

cpt102 数据结构与算法

目录

L1 Introduction to data structures	3
Data structure 数据结构	3
Arrays	4
栈与队列 (Stacks and Queues)	5
L2 Introduction to Algorithms	10
Algorithms	10
课程中主要使用的表示方法	11
Pseudocode	11
Time complexity	14
L3 List-based Data Structures	16
Abstract data types	16
The List ADT	19
List-based data structures	20
各 ADT 选用对应主流存储的底层原因 (硬件+算法双重逻辑) 	25
L4 Recursion	26
Recursive algorithms	27
迭代 vs 递归: 优缺点对比	31
L5 Searching, sorting	32
查找算法	32
排序算法	33
排序算法定义	34
Bubble sort	35
Insertion sort	35
Selection sort	36
L6 Faster sorting	36
分治法 (Divide-and-Conquer)	36
归并排序 (Merge Sort)	36

快速排序 (Quick Sort)	37
原地和稳定	38
复杂度分析工具 lower bounds and Big-Theta	38
主定理的使用步骤 (考试 / 刷题秒用)	39
非比较类排序	40
流程 (LSD 低位优先)	41
L8 Union Find	42
Union-Find	42
1. ListOfSetsUF (朴素实现)	42
2. QuickFind (快速查找)	43
3. QuickUnion (快速合并)	43
4. Weighted Quick Union (加权快速合并, 最优)	43
Queue and Binary Heap	43
Binary Heaps	45
二叉堆的数组表示	45
L9	47
Tree	47
树的遍历 (Tree Traversal)	47
Graph	48
L10	55
SPT Shortest Path Tree	55
Dijkstra 算法--SPT 单源最短路径 (非负权边)	55
MST Minimum Spanning Trees	58
关于 MST 和 SPT	58
Cut Property 割性质	58
Generic MST Finding Algorithm 通用 MST 算法 一个算法框架	59
Prim's Algorithm	60
Kruskal's Algorithm	61
L11 图问题求解	64
无向图连通分量 (Connected Components)	64
拓扑排序 (Topological Sort) --DAG 排序算法	66
Strongly-connected Components ---有向图	67
L12 Binary Search Trees 二叉搜索树	69

Rooted Binary Trees	71
BST 键的排序规则（必须满足的三大性质）	72
2-3 Trees	73
左倾红黑树（Left-Leaning Red-Black BSTs, LLRB）	76
L13	77
Hashing	78

分数构成：

ict1-10% week7

ict2-10% week12

期末考试-80%

L1 Introduction to data structures

Data structure 数据结构

Def:A data structure is a specialised format for organising, storing, and managing (aggregate) data in a computer so that it can be used efficiently.

数据结构是一种专用格式，用于在计算机系统中高效地组织、存储和管理聚合数据。

A structure which stores aggregate data is sometimes also referred to as a collection or container.这些结构也称为集合或容器，用于容纳数据元素并允许对其高效地执行操作。

-为什么需要不同类型的数据结构？不能只用 arrays 吗？

-理论可行但是实际应用存在短板

Potential memory leaks or waste; 内存泄漏或空间浪费

Inefficient for certain operations (e.g., searching); 时间复杂度很高，效率低下

Lack the richness to express more complex, highly structured, data (e.g., graphs) 难以高效地表达和处理更复杂、高度结构化的数据

如何选择正确的数据结构 Selecting the correct data structure

Two key considerations:

维度 1：数据结构的内在属性 (Intrinsic Properties)

重复性约束： 场景是否允许数据重复(duplicated data)

eg: 统计用户访问记录需保留重复数据, 而存储用户唯一 ID 则需去重

顺序性约束: 数据是否需要保持有序 / 无序、排序 / 未排序(ordered/unordered? sorted/unordered)

eg:有序 / 无序: 如消息队列需保证消息的时序性 (有序), 而用户标签集合则无需固定顺序 (无序)。

排序 / 未排序: 如学生成绩榜单需实时排序, 而临时缓存的订单号则无需排序。

约束的强制性: 不同数据结构自带固定约束, 选型时需先匹配这些约束, 而非强行突破。force different constraints.

示例: 集合 (Set) 天然去重, 数组天然有序, 链表则灵活支持插入删除但无序。

维度 2: 待执行的典型操作 (Typical Operations)

该维度聚焦**操作效率**, 是选型的核心依据, 核心逻辑如下:

操作类型: 需明确核心操作是迭代(iteration)、新增、删除、随机访问、查找中的哪一种或几种。

效率差异: 不同数据结构对同一操作的时间复杂度 (效率) 差异显著

高频优先原则: 优先保障**最频繁执行的操作**的效率, 以此确定核心数据结构。

示例: 若高频操作是“随机访问”, 优先选数组; 若高频操作是“插入 / 删除”, 优先选链表; 若高频操作是“查找”, 优先选哈希表。

Data structures 分类 (数据模型+算法)

List-based structures such as ArrayList, LinkedList, Stack, Queue;

Also called linear data structures; 线性数据结构--数据元素按线性顺序排列, 是最基础的结构类型。

Tree-based structures such as Heap(堆), Binary Search Tree (二叉搜索树), AVL (Adelson-Velsky and Landis) Tree (平衡二叉搜索树); --数据元素以层级关系组织, 能高效支持查找、插入、删除等操作。

Graphs and Networks; --用于表示元素之间多对多的复杂关系, 是描述现实世界网络 (如社交网络、交通网络) 的理想模型。

Hashing-based structures such as HashSet, HashMap.--通过哈希函数实现近乎常数的查找、插入和删除操作。

Arrays

DEF: An array is a simple ordered (linear) collection of data.

Arrays store their elements in *consecutive memory* (连续内存块) locations and refer to them by their index. ----**O (1)** 时间复杂度的随机访问

contiguous blocks of memory in a computer. 数组在物理内存上是一块完整的区域。

An array has a fixed number of elements, which is called its size or length. 数组包含的元素数量是**固定的**

数组操作 (Array operations)

1. 迭代 (Iteration) 数组的迭代严格按照**索引顺序**进行
2. 索引操作 (Indexing)

通过索引直接检索或修改指定位置的值, 时间复杂度为 **$O(1)$** , 这是数组最核心的优势。

数组的索引机制支持丰富的位置相关操作 (随机访问 子数组提取 按索引查找)

3. 核心限制: 固定长度 数组在创建时长度就已确定, **无法直接添加或删除元素**

数组的优缺点 (Pros and Cons)

极快的随机访问: 数组是连续的内存块, 这使得通过索引访问元素的速度非常快, 时间复杂度为 **$O(1)$**

原理: 本质上就是“直接跳到”目标位置, 无需遍历, 因此访问速度极快。

1. 获取数组第一个元素 `array[0]` 的内存地址。
2. 将该地址加上 `i × 单个元素的内存大小`, 即可直接定位到第 `i` 个元素 `array[i]`。

查找效率低: 在数组中进行查找时, 通常需要遍历整个数组, 时间复杂度为 **$O(n)$** , 速度很慢

增删操作浪费资源: 由于数组长度固定, 若要添加或删除元素, 必须创建一个新数组, 并将所有元素复制过去。

栈与队列 (Stacks and Queues)

Stacks and queues are two array-based (or list-based) data structures.

They are **resizeable** 可调整大小: elements can be added and removed. 元素一旦被处理, 就会从结构中移除

栈 (Stack): 遵循 **后进先出 (Last In, First Out, LIFO)** 原则

队列 (Queue): 遵循 **先进先出 (First In, First Out, FIFO)** 原则

Stacks (LIFO)

A stack is a linear (list-based) data structure in which elements may be inserted or removed from only one end. 栈是一种线性 (基于列表) 的数据结构, 元素只能从**一端**进行插入或移除操作。

栈支持三个核心操作, 所有操作都只在栈顶进行:

- **push(e)**: 将新元素 **e** 添加到栈顶。
- **pop()**: 移除并返回栈顶元素。如果栈为空，此操作通常会报错。
- **peek()**: 返回栈顶元素，但不将其移除，用于查看当前栈顶的值。
- 通常还会提供一个 **size()** 操作，用于返回栈中当前的元素数量，方便判断栈是否为空或已满。

这三个操作的时间复杂度均为 **O(1)**，速度极快。except when resizing is required (too much or not enough free space at the top).唯一的例外是当栈需要**调整大小**时（例如，底层数组已满需要扩容，或空间过多需要收缩），此时会涉及内存复制，性能会有所下降。

应用：

处理结构化（嵌套）数据文件 Processing files of structured (nested) data

E.g., reading files with structured markup (HTML, XML, etc.)

原理：利用栈来跟踪嵌套的标签（如 `<div>` 内的 `<p>`），遇到开标签入栈，遇到闭标签出栈，以此验证标签的匹配性和层级结构。

程序执行与函数调用 Program execution, e.g. working on subtasks, then returning to previous task.

原理：当程序执行一个函数时，将当前上下文（返回地址、局部变量等）压入栈；函数执行完毕后，从栈顶弹出上下文，恢复到调用前的状态，实现“先调用后返回”的逻辑。

Undo in editors 编辑器的撤销

原理：每一次编辑操作（如输入、删除）都被压入栈中；当用户点击“撤销”时，从栈顶弹出最近一次操作，并恢复到之前的状态。

Expression evaluation 表达式求值

队列 (Queue)

Def: A (standard) queue is a linear data structure in which elements are inserted only at one end, and removed only from the other end.

一个（标准）队列是一种线性数据结构，元素只能从**一端插入**，并从**另一端移除**。

先进先出 (FIFO)：遵循“先进入队列的元素，先被处理”的原则。

双端开放模型：可以用数组来模拟队列，其中两端都是开放的。

队列 (Queue) 的核心操作

enqueue(e)：将新元素 **e** 添加到队列的**尾部 (tail)**。

dequeue(): 移除并返回队列 ** 头部 (head) ** 的元素。如果队列为空, 此操作通常会报错。

peek(): 返回队列头部的元素, 但不将其移除, 用于查看当前队首的值。

通常还会提供一个 **size()** 操作, 用于返回队列中当前的元素数量, 方便判断队列是否为空或已满。

这三个操作的时间复杂度均为 **O(1)**, 速度极快。

唯一的例外是当队列需要**调整大小 (resizing)** 时 (例如, 底层数组已满需要扩容, 或空间过多需要收缩), 此时会涉及内存复制, 性能会有所下降。

在 Java 中, 常用 **offer()** 和 **poll()** 来实现入队和出队操作。

队列 (Queue) 的典型应用场景

队列的核心价值在于**按序暂存并处理元素**。Most commonly used when some task/program/system needs to hold elements prior to processing.

- **Task scheduling for Operating Systems.** 操作系统任务调度

CPU 进程调度: 按时间片轮转或优先级调度待执行的进程。

打印假脱机 (Print Spooling): 将多个打印任务按提交顺序排队, 依次送往打印机。

- **Programs which simulate/represent queues in the real world.** 模拟现实世界的排队场景

- **Some algorithms that we will see later in this course use queues** 算法

Eg: 广度优先搜索 (BFS)

- **Applications that require concurrency and multi-threading.** 并发与多线程应用

Java 集合框架 (Java Collection Framework)

Java 集合框架是一个**用于表示和操作集合的统一架构**, 它提供了一套标准的接口和实现, 让开发者可以高效地处理数据。

- **接口 (Interfaces)**: 抽象数据类型, 定义了不同容器的行为, 如 **List**、**Set**、**Map** 等。

- **实现类 (Implementations)**: 接口的具体实现, 如 **ArrayList**、**HashSet**、**HashMap** 等, 它们提供了接口定义的具体功能。

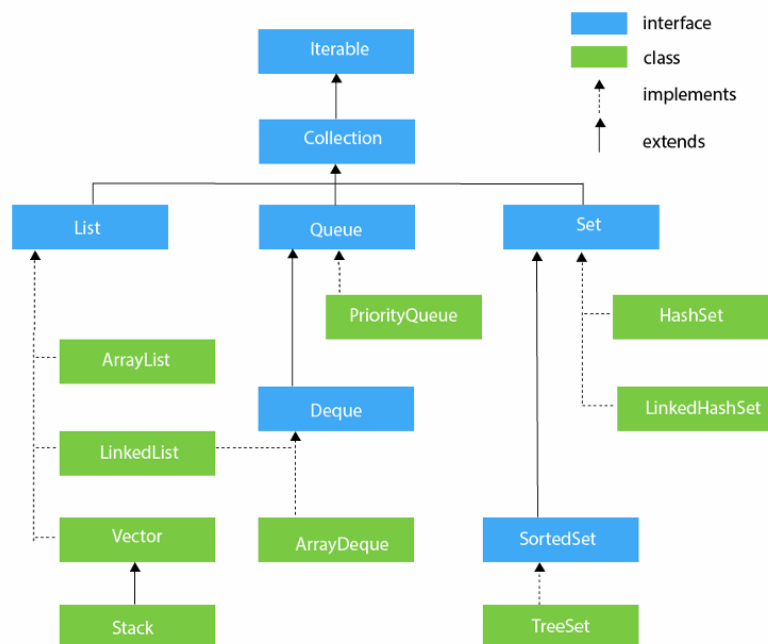
- **算法 (Algorithms)**: 对集合进行操作的多态方法, 如查找、排序、添加等。这些方法名称相同, 但具体实现会根据集合类型的不同而变化。

集合框架主要分为两大类:

1. **Collection**: 用于处理单一值的集合, 如列表、集合、队列等。

2. Map：用于处理键值对（Key-Value）的映射关系。

Java Collection 层级结构解读



What do these words “interface”, “implements” mean??

Java 接口（Interface）

接口是类的**蓝图（blueprint）**或**抽象框架（abstract framework）**。

一个类通过 **implements** 关键字来“实现”一个接口，从而遵守该接口的契约。

接口的作用：契约（Contract）

- 接口为实现它的类提供了一份必须遵守的**契约**。
- 它通过定义一组方法签名，规定了实现类“必须能做什么”，但不规定“如何去做”。

接口的灵活性

- 一个接口可以被**很多不同的类**实现。
- 例如，**List** 接口就有 **ArrayList**、**LinkedList** 等多种实现，它们都遵守了 **List** 的契约，但内部实现方式完全不同。

核心原则：“做什么” vs “怎么做”

- 接口只定义**做什么（what）**：规定了类应该具备的行为。
- 实现类负责**怎么做（how）**：提供了这些行为的具体逻辑。

方法与行为的关系

- 一个类或对象“能做什么”，是由它内部的 **** 方法（methods） **** 来定义的。

- 接口只规定了“能做什么”（方法签名），但完全不涉及“怎么做”（没有实现细节）。

抽象方法（Abstract Method）

- 没有实现体(unimplemented)的空方法被称为**抽象方法**。
- 接口的核心就是由这些抽象方法组成的。

接口的正式定义

- **Java 接口是一组抽象方法的集合，通常数量不多。**
- 它只定义了行为的契约，不提供任何实现。

实现接口的强制要求

- 当一个类 `implements`（实现）一个接口时，它 **** 必须重写（override）**** 接口中的所有抽象方法，并提供具体的实现逻辑。
- 否则，这个类必须被声明为 `abstract`（抽象类）。

接口的语法

```
public interface Writable {  
    public static final int MAX_LINES = 1000;  
    public boolean write(String fileName);  
}
```

接口中的方法默认是 `public` 和 `abstract` 的，因此 `public` 关键字是可选的。

The `write()` method is abstract: it has no body, only a signature followed by a terminal semi-colon.

接口中只能定义 `public static final` 类型的常量，这些关键字也是可选的。

接口不能有实例变量（非静态成员变量）。

Collection 接口

`Collection` 是整个集合层级结构中的**根接口 root interface in the collection hierarchy**.

A collection is a group of objects (elements). 它代表了“一组对象（元素）”的抽象概念。

多样的实现

- 不同的集合实现（如 `ArrayList`、`HashSet`、`LinkedList`）在特性上各不相同：
 - 有的允许重复元素，有的不允许；
 - 有的是有序的，有的是无序的。

- 但它们都遵守 `Collection` 接口定义的契约。

通用方法

`Collection` 定义了所有集合都应该具备的核心方法，例如：

- `add()`：添加元素
- `remove()`：移除元素
- `contains()`：判断是否包含元素
- `size()`：返回元素个数
- `isEmpty()`：判断是否为空

`Collection` 接口继承了 `Iterable` 接口，这意味着所有的集合都可以被迭代，我们可以使用 `for-each` 循环来遍历集合中的每一个元素。

L2 Introduction to Algorithms

Algorithms

DEF: An algorithm is a finite sequence of precise and well-defined instructions to solve a specified problem. 算法是一组有限的、精确且定义明确的指令序列，用于解决特定问题。

instructions are “precise and well-defined”-----> must be unambiguous, easy to follow.

The instructions form a “finite sequence”----->

序列 (sequence)：步骤必须按正确的顺序执行，不能乱序。

有限 (finite)：算法必须在有限步骤后终止，不能无限运行下去。

算法的目标就是解决特定问题，解就是算法的 output,要处理的数据就是要接收的 input

Representation of algorithms

- **用代码 (In code)**：
 - 优点：精确、可直接运行。
 - 缺点：依赖具体编程语言（Java、Python 等），不利于跨语言理解算法核心思想。
- **用自然语言 (In plain English)**：
 - 优点：容易上手，直观易懂。
 - 缺点：容易出现歧义或描述不精确，导致理解偏差。

课程中主要使用的表示方法

为了兼顾精确性和通用性，这门课主要使用两种方式来表示算法：

- **伪代码 (Pseudocode)**：一种类似代码的半结构化语言，不依赖具体编程语言，既精确又易读。
- **流程图 (Flowcharts)**：用图形化的方式展示算法的执行流程和分支，非常直观。

Pseudocode

DEF: Pseudocode is a notation system used to describe the steps in an algorithm using a mixture of natural language and high-level programming constructs.

伪代码是一种符号系统，它混合了自然语言和高级编程结构来描述算法的步骤。

特点：

Intended to be read by humans, not computers. 面向人类阅读

More structured than plain English.

Easy to convert into actual computer code, only need to translate line-by-line.

Notation system is not universal

它的核心优势在于**语言无关性 (language-independence)**：用一种足够精确的符号来表示算法，既不依赖具体编程语言，又能很方便地转换成实际代码，完美地平衡了精确性和通用性。

Pseudocode example: simple

Algorithm 1: Linear array search

Input: An array of integers `arr`; an integer to search `val`

Output: The index of first occurrence of `val` in `arr` if it exists, -1
otherwise

```
 $n \leftarrow \text{len}(\text{arr});$ 
for  $i \leftarrow 0$  to  $n - 1$  do
    if arr[i] = val then
        return  $i$ ;
return -1;
```

Pseudocode example: more complex

Algorithm 2: Checking if expression is well-balanced

Procedure ISWELLBALANCED(expr)

Input: a String expr

$S \leftarrow \text{Stack}();$

▷ S is an empty Stack

$i \leftarrow 0;$

while $i < \text{len}(\text{expr})$ **do**

$c \leftarrow \text{expr}[i];$

▷ c is the i -th character of expr

if c is an opening delimiter **then**

$S.\text{push}(c);$

else if c is a closing delimiter **then**

if $S.\text{isEmpty}()$ **then**

return false;

else

$c' \leftarrow S.\text{pop}();$

if c and c' do not match **then**

return false;

$i \leftarrow i + 1;$

return $S.\text{isEmpty}();$

检查括号平衡算法的伪代码实现

伪代码的书写指南

- Mimic good code and good English.
 - Use descriptive variable names;
 - Include comments where useful;
 - English should be comprehensible (full sentences not needed).
- Ignore unnecessary details, omit information that is clear from context.
 - E.g., types of variables are often obvious.
- Ultimate goals of pseudocode:
 - Easy to understand;
 - Easy to convert into actual code;
 - Easy to analyse the underlying algorithm.

1. 模仿好的代码和好的英文

- 使用有描述性的变量名: 比如用 `stack` 而不是 `s`, 用 `currentChar` 而不是 `c`, 让代码自解释。
- 在需要的地方加注释: 注释不是越多越好, 只在能提升理解时添加, 比如说明某个变量的用途或特殊逻辑。
- 英文要易懂: 不需要写完整的句子, 用短语即可, 但必须清晰, 避免歧义。

2. 忽略不必要的细节

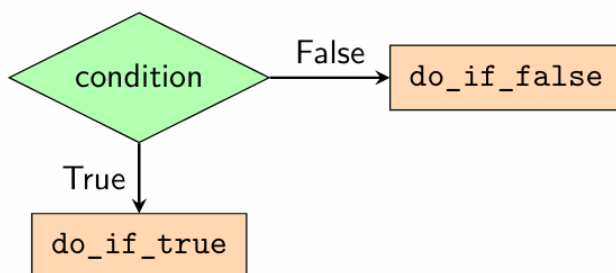
- 省略那些从上下文就能看出来的信息, 比如变量的类型 (`int`, `String`), 因为伪代码的核心是逻辑, 不是语法。
- 这样可以使伪代码更简洁, 专注于算法的核心步骤。

3. 伪代码的终极目标

- 易于理解: 任何人(有编程经验的)都能快速看懂逻辑。
- 易于转换为实际代码: 结构清晰, 逐行翻译即可。
- 易于分析底層算法: 能清晰地看到算法的时间复杂度、空间复杂度等核心特性, 便于分析和优化。

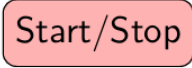
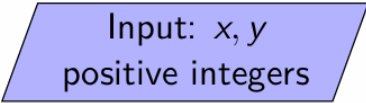
Flowcharts

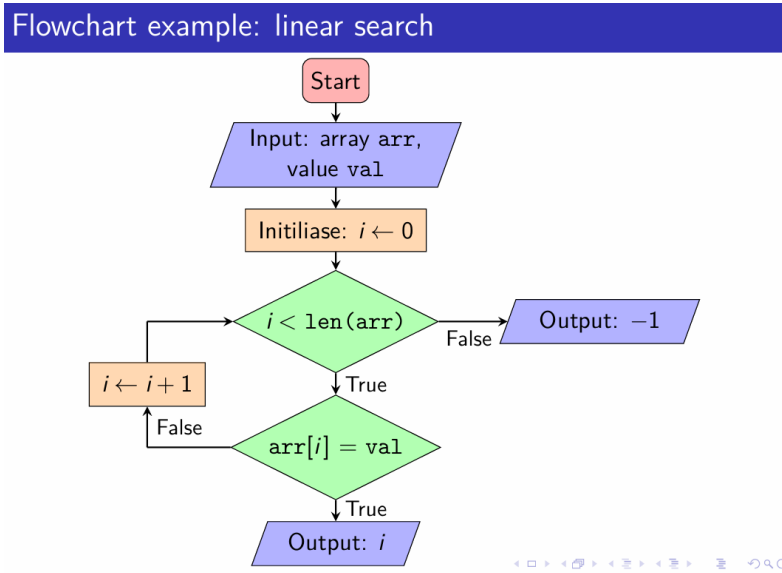
DEF: A flowchart is a notation system used to represent processes in visually appealing, easy-to-follow, form. 流程图是一种符号系统, 用视觉化、易于跟随的形式来表示流程 (process)



判断块 (Decision Block)

- **形状**: 菱形 (diamond)
- **用途**: 表示条件判断, 用于 `if/else` 结构和 `while` 循环。
- **限制**: 通常只支持简单的二分支 (`if/else`), 不直接支持 `else if`。
- **标注惯例**:
 - 条件为 `True` (或 `Yes`) 时, 箭头通常向下指向 `do_if_true`。
 - 条件为 `False` (或 `No`) 时, 箭头通常向侧面指向 `do_if_false`。
 - 有时会写成问句形式 (如 `condition?`), 此时分支标注为 `Yes/No`。
- **处理块 (Process Block)**
- **形状**: 矩形 (rectangle)
- **用途**: 表示具体的操作或语句, 比如赋值、计算、函数调用等。
- **作用**: 是算法中“做事”的步骤。

- Beginning and end blocks:  **Start/Stop** (rounded rectangle)
- Input and output blocks:  **Input: x, y
positive integers** (parallelogram)



什么是好的算法?

一个“好算法”通常是多种优秀特质的结合, 而不是单一维度的最优:

- **简洁 (Simple)**: 易于表达和理解, 逻辑清晰, 便于维护和修改。

- **快速 (Fast)**: 能在短时间内终止, 即时间效率高。
- **内存高效 (Memory-efficient)**: 优化内存使用, 空间效率高。
- 通常来说, “**All of the above**” 是理想目标, 但在实际中往往需要根据场景在这些维度之间做权衡。

算法的终止速度受很多外部因素影响

- The language it is implemented in;
- The computer it is executed on (and its state, e.g. how many other tasks are running at the same time);
- The algorithm's input;

我们转而测量**算法需要执行的操作数量**, 这个指标被称为 **(时间) 复杂度 ((time) complexity)**。

Time complexity

DEF: The time complexity of an algorithm measures the amount of elementary operations that need to be performed in an execution of the algorithm

赋值 (Assignments): 如 `i ← 0`

比较 (Comparisons): 如 `if arr[i] = val` 或 `while i < n`

算术运算 (Arithmetic operations): 加、减、乘、除、取模等

返回 (Returns): 如 `return i` 或 `return -1`

以及其他类似的基础操作。

最坏情况 (Worst-case) 和 **平均情况 (Average-case)**。

线性查找的操作次数分析

以线性查找算法为例, 它的操作次数完全取决于输入:

- **最好情况 (Best-case)**: 目标值 `val` 是数组的第一个元素 `arr[0]`
 - 操作: `n ← len(arr)` (赋值) + `i ← 0` (赋值) + `arr[0] = val` (比较) + `return 0` (返回)
 - 总共 **4 次操作**, 与数组大小 `n` 无关。
- **最坏情况 (Worst-case)**: 目标值 `val` 不在数组中, 或者在最后一个位置
 - 操作: 2 次赋值 + `n` 次比较 + 1 次返回

- 总共 $n + 2$ 次操作，与数组大小 n 线性相关。

最坏情况 (Worst-case): 算法在最不利输入下的表现，即“能有多糟？”。这是我们评估算法可靠性的主要依据，因为它保证了算法在任何情况下都不会比这个更慢。

平均情况 (Average-case): 算法在“典型”或“随机”输入下的表现，即“通常情况下有多快？”。这能更真实地反映算法在实际中的表现。

Worst- and average-case time complexities

The worst-case time complexity of an algorithm is the function $T_{\text{worst}} : N \rightarrow N$ such that, for all $N \in N$, $T_{\text{worst}}(N)$ is the maximal time complexity of the algorithm over all inputs of size N .

算法的最坏情况时间复杂度是一个函数 $T_{\text{worst}}:N \rightarrow N$ ，对于所有输入规模 $N \in N$ ， $T_{\text{worst}}(N)$ 是该算法在**所有规模为 N 的输入上**所表现出的最大时间复杂度。

什么是“输入规模 N ”？

这个概念的含义通常由上下文清晰定义：

- 如果输入是一个集合（如数组、列表）， N 就是集合中元素的数量。
- 如果输入是一个正整数， N 就是这个整数本身的值。
- 如果有两个不同规模的输入，我们会写成 $T(M, N)$ 来分别表示。

The average-case time complexity of an algorithm is the function $T_{\text{avg}} : N \rightarrow N$ such that, for all $N \in N$, $T_{\text{avg}}(N)$ is the expected time complexity of the algorithm when given a random input of size N .

平均情况时间复杂度 $T_{\text{avg}}(N)$ 是：在**所有规模为 N 的“随机输入”**下，算法执行时间的期望值 (**Expected Time**)。

Average-case analysis: linear search

Model: Entries in our input array are i.i.d. random integers, each with a fixed probability p of being equal to the value searched for.

- Let us denote $p_k := \mathbb{P}(\text{LINEARSEARCH}(\text{arr}, \text{val}) = k)$ the probability that the `LINEARSEARCH` algorithm returns k , for $k \in [0, N-1]$, and p_{FAIL} the probability that it returns -1 (searched value not found).
- Then we have: $T_{\text{avg}}(N) = \sum_{k=0}^{N-1} p_k \cdot \tilde{T}(k) + p_{\text{FAIL}} \cdot \hat{T}(N)$.
- Moreover, in our model, $(p_k)_{0 \leq k \leq N-1}$ is just a truncated geometric distribution, i.e.: $p_k = (1-p)^k \cdot p$ for $k \in [0, N-1]$.
- Finally, we have $p_{\text{FAIL}} = (1-p)^N$.

Bringing it all together, and using previous results, we get:

$$T_{\text{avg}}(N) = p \cdot \sum_{k=0}^{N-1} (1-p)^k (3k+5) + (1-p)^N (3N+4)$$

Big-Oh notation

Definition

Let $f, g : \mathbb{N} \rightarrow \mathbb{R}$ be two functions from the set of non-negative integers to the set of (strictly) positive real numbers. We say that “ $f(N)$ is Big-Oh of $g(N)$ ” and write $f(N) = O(g(N))$, if there exists some constant $C > 0$ such that $f(N) \leq C \cdot g(N)$ for all $N \in \mathbb{N}$.

Complexity estimates

Objective

For a complexity $T(N)$, look for the simplest function $g(N)$ such that $T(N) = O(g(N))$, and where g is a “close estimate” for $T(N)$.

Typical scales, in increasing order.

- Constant: $g(N) = 1$.
- Logarithmic: $g(N) = \log(N)$.
- Sub-linear: $g(N) = N^a$ for some $0 < a < 1$.
- Linear: $g(N) = N$.
- Log-linear: $g(N) = N \cdot \log(N)$.
- Quadratic: $g(N) = N^2$.
- Polynomial: $g(N) = N^k$ (usually $N > 1$).
- Exponential: $g(N) = a^N$ for some $a > 1$.

Estimating complexities with Big-Oh notation is usually referred to as **asymptotic analysis** of algorithms.



Space complexity

Definition

The **space complexity** of an algorithm measures the amount of *auxiliary* memory space required for an execution of the algorithm.

L3 List-based Data Structures

Abstract data types

Definition

An **abstract data type** (ADT) is a specification of a data type, independent of an implementation. It consists of a type name and a set of specified operations.

仅指定**类型名**和**操作集合**的数类型规范，无实现细节，操作需明确前置 / 后置条件（即操作契约）。

核心价值

- ADTs allow the programmer to think more generally and conceptually 帮助程序员更通用、更概念化地思考

Start with most general properties required for our data structure without worrying about implementation details.

Reduces cognitive load. 降低认知负荷

- Language-independence 语言无关性
- ADTs provide flexibility 提供代码灵活性（在同一门语言内）

ADT 是「功能说明书（抽象规范）」，数据结构是「落地实施方案」；数据结构用来实现 ADT

Java 集合框架（Java Collection Framework, JCF）

“a unified architecture for representing and manipulating collections” 统一的架构

接口 (**Interfaces**) = ADT 的契约 represent different sorts of containers; List, Set, Map, etc.

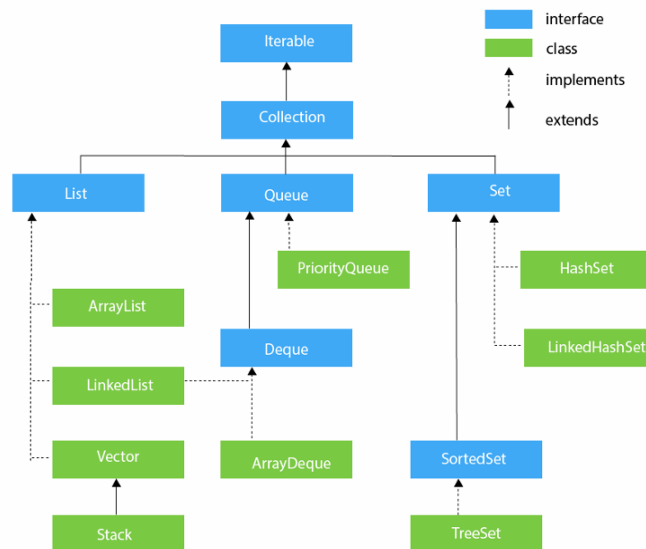
实现 (**Implementations**) concrete realisations of the ADTs ;ArrayList, HashSet, HashMap, etc

算法 (**Algorithms**)

operations (methods) that manipulate the collections and perform useful operations on them (search, sort, add, etc.); 操作集合的通用方法

the operation is specified on the ADT, and its implementation depends on the concrete realisation. 算法是在 **ADT 接口** 上指定的，但具体执行代码 **依赖于具体的实现类**

The Collection hierarchy



The Collection interface

The root interface in the collection hierarchy Java 集合框架的根接口

A collection is a group of objects, known as elements in the collection. 它代表一组对象，这些对象被称为集合的 **元素 (elements)**

Different implementations: with/without duplicates, ordered/unordered, etc.. 不同实现类可以有不同特性：允许重复 / 不允许重复、有序 / 无序等。

- Collection operations: `add()`, `remove()`, `contains()`, `size()`, `isEmpty()`, and a few others; see here.
 - Most general operations for a collection of objects. Any collection of objects should have these.
 - But how they are *implemented* depends on the type of collection.

Collection extends Iterable(迭代) ---所有 Collection 的实现类都支持 迭代

(Iterate)：可以逐个遍历集合中的元素

The Collection interface has no direct implementation Collection 接口没有直接实现类，所有具体类都实现它的三个子接口：List、Queue、Set

- List: an ordered collection of elements, with possible duplicates. 有序集合，允许重复元素
- Queue: a collection for holding elements prior to processing; generally first-in first-out (FIFO) 处理前暂存元素的集合，通常先进先出
- Set: an unordered collection of elements, without duplicates 无序集合，不允许重复元素

The List ADT

Definition

A **list** is a linear, ordered ADT. It consists of a finite sequence of data items known as *elements*, ordered by *index* or *position*.

Additional constraints (限制):

Addition: `add(element)` adds to the tail of the list (highest index) `add` 是 添加到尾部

Deletion: `remove(element)` removes the first (by index) occurrence of element.
`remove` 删除第一个

Iteration: order follows indexing, from head to tail 迭代遵循索引顺序

List-specific operations (专属操作):

1.1 位置访问 (Positional Access)

- `get(index)`: 根据索引获取元素。
- `set(index, element)`: 根据索引替换 / 修改元素。
- 特点: `ArrayList` 实现此操作极快 ($O(1)$), 因为是直接内存寻址。

1.2 位置搜索 (Positional Search)

- `indexOf(element)`: 查找元素第一次出现的索引。
- `indexOf(startIndex, element)`: 从指定位置开始查找元素的索引。
- `lastIndexOf(element)`: 查找元素最后一次出现的索引。
- 特点: 本质是遍历查找, 时间复杂度通常为 $O(n)$ 。

1.3 位置修改 (Positional Modification)

- `insert(index, element)`: 在指定索引位置插入元素 (后续元素会后移)。
- `remove(index)`: 根据索引删除元素 (后续元素会前移)。
- 特点: 这是 List 的核心能力之一, 但 `ArrayList` 实现时涉及数组元素移动, 效率为 $O(n)$ 。

1.4 范围视图 (Range-view)

- `subList(startIndex, endIndex)`: 截取当前列表的一段 (从 `start` 到 `end-1`), 返回一个新的 List 视图。
- 特点: 方便对集合的局部进行操作, 是代码复用的重要手段。

ADTs: conceptual specification = name + specified operations 概念规范 = 名字 + 指

定操作

Data structures: concrete implementations/realisations of ADTs. ADT 的具体实现 = 数据模型 + 算法

A data structure includes:

Data model = how the structure stores the data; 数据怎么存

Algorithms = implementations of the specified operations. 操作如何实现

与 OOP 编程对比:

数据结构的模型 = 面向对象编程中的类 (**Class**)

类 = 成员变量 (数据) + 成员方法 (操作 / 算法)

List-based data structures

List ADT 的两种经典实现: 动态数组 (**ArrayList**) 和 链表 (**LinkedList**)

1. ArrayList: resizable array implementation

Data model is an array with additional space for new elements. “预留空间”

2. LinkedList: stores elements through dynamic allocation of link nodes.

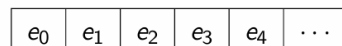
单链表 (Singly-linked): 只有后继指针-----唯一指向 (只知道下一位是啥)、无头无尾 (需要 head 找到第一个节点再依次遍历)

双链表 (Doubly-linked): 既有后继指针 `next`, 又有前驱指针 `prev` (Java 的 `LinkedList` 就是这种), 支持双向遍历。

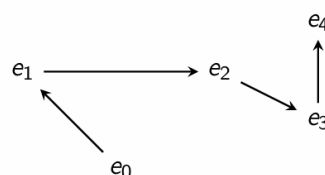
循环链表 (Circular): 尾节点指向头节点, 形成环。

ArrayList vs LinkedList

- ArrayList is implemented as an array: contiguous block of memory, with extra capacity at the end (for new elements).



- Fast access (retrieval) and iteration.
- When inserting/deleting, need to shift elements.
- LinkedList elements can be anywhere in memory. Each element is stored inside a *node* which also holds a *pointer* to its *successor* in the list.



- Can be faster for insertion and deletion.
- Access needs to traverse the list.

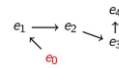
ArrayList complexity: get(k)

For ArrayList (or array[k]): variable just stores reference to memory address of initial element.

- First, access initial element at head of ArrayList.
Retrieving a reference = constant $O(1)$ time. *第k个 = 首地址 + k * 单元大小*
- Then, because arrays are contiguous in memory, just need to *offset* by k (see Week 1).
Equivalent of an operation "+ = k", so constant $O(1)$ time.
- Total complexity: $O(1)$. *按地址计算位置 → O(1)*

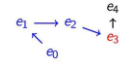
LinkedList complexity: get(k)

- First, access initial element at head of LinkedList. *从表头遍历*



Retrieving a reference = constant $O(1)$ time.

- Then, need to *traverse* the list until we get to k -th element, i.e. visit elements one-by-one.



- Complexity = $O(k)$
- Average- and worst-case complexities are $O(n)$ (linear).

Array-based data structure model

ArrayStructure: `E[] elements; int size;`

E is the type of the elements in our data structure, meaning our data structure is generic (泛型)

数组 `elements` 中, 恰好前 `size` 个元素是非空的, 后面的位置是预留的空闲空间 (用于后续添加元素)

内置 `CHECKCAPACITY()` 过程: 在添加元素前检查数组是否还有剩余空间;

如果空间不足, 调用 `RESIZE()` 过程:

- 申请一个更大的新数组;
- 把旧数组的所有元素复制到新数组;
- 让 `elements` 指向新数组。

RESIZE() 复杂度: 时间和空间都是 $O(n)$ (复制 n 个元素), 但极少被调用 (所以 `add()` 操作平均仍可视作 $O(1)$)。

Singly-linked list model

`SinglyLinkedList<E>`:

`Node<E> head;` // 链表头节点 (第一个元素)

`Node<E> tail;` // 链表尾节点 (最后一个元素)

`int size;` // 元素总数

`Node<E>`:

`E data;` // 存储元素数据

`Node<E> next;` // 指向后继节点的指针

节点 (Node) 的作用

- 每个节点封装两个东西:

- **E data**: 真正要存的元素 (泛型, 可存任意类型);
- **Node<E> next**: 指向链表中“下一个节点”的引用, 这是链表串联的关键。

链表维护 **head (头)** 和 **tail (尾)** 两个指针, 方便快速访问两端;

从 **head** 出发, 连续调用 **next** 指针 **size-1** 次, 最终会到达 **tail**:

`head.next.next...(size-1 次) = tail`

尾节点 **tail** 的 **next** 必须为 **null** (表示链表结束, 没有后继节点)

`tail.next = null`

Pseudocode for get()

For ArrayList:

```
Procedure GET(list, k)
  return list.elements[k];
```

For SinglyLinkedList:

```
Procedure GET(list, k)
  i ← 0, current ← list.head;
  while i < k do
    current ← current.next;
  return current.data;
```

初始化变量
从 head 开始

- When implementing in code, you should also do a bounds check $0 \leq k < \text{list.size}$.
- Complexity analysis is straightforward, we get:
 - Time complexity: $O(1)$ for ArrayList, $O(n)$ for SinglyLinkedList;
 - Space complexity: $O(1)$ for both.

ArrayList complexity: add(element)

- Recall that for list-based structures, `add()` adds (appends) the new element at the tail of the list.
- First, access empty space at tail of ArrayList.

e_0	e_1	e_2	e_3	e_4	0	...
-------	-------	-------	-------	-------	---	-----

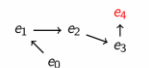
 Like `get()`, constant time $O(1)$.
- Then, just need to put new element in first empty space.

e_0	e_1	e_2	e_3	e_4	e_5	...
-------	-------	-------	-------	-------	-------	-----

 Equivalent of a "set" operation (`array[k] = e`), so $O(1)$.
- Total complexity: $O(1)$.
- BUT!!! If no space left, i.e. have reached *capacity*, need to copy all the array into a new space using `RESIZE()`, so $O(n)$.

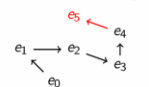
LinkedList complexity: add(element)

- First, access initial element at tail of LinkedList.



Because LinkedList stores references to head and tail, this is $O(1)$.

- Then, just create new link to new element e_5 .



Creating a link is just assigning a variable, so $O(1)$.

- Total complexity: $O(1)$.

Procedure ADD(list, element)

`CHECKCAPACITY(list);` // ► 可能需要扩容 (RESIZE)

`list.elements[size] ← element;` // 把元素放进尾部空闲位置

`list.size ← list.size + 1;` // 元素个数+1

Procedure ADD(list, element)

`new_tail_node(data=element, next=null);` // ► 创建新节点, 作为新尾节点

```
list.tail.next ← new_tail_node; // ▶ 旧尾节点指向新节点

list.tail ← new_tail_node;    // ▶ 更新链表尾指针为新节点

list.size ← list.size + 1;     // 元素个数+1
```

ArrayList complexity: insert(index, element)

- First, access k-th element of ArrayList.

Like get(), constant time $O(1)$.
- Then, need to insert new element, which means shifting all remaining elements by one place.

Complexity = number of elements to shift = $O(n - k)$.
- Average and worst cases are both $O(n)$.

```
Procedure INSERT(list, index, element)
  CHECKCAPACITY(list);
  k ← list.size;
  while k > index do
    list.elements[k] ← list.elements[k - 1];
  list.elements[index] ← element;
  list.size ← list.size + 1;
```

LinkedList complexity: insert(index, element)

- First, access k-th element of LinkedList.

Like get(k), so $O(n)$.
- Then, just break old link $e_2 \rightarrow e_3$, and create new link to/from e_5 instead.

All this is constant time $O(1)$.
- Total complexity is therefore $O(n)$.
 - BUT much faster if $k \ll n$.

对于 insert() 操作：

- ArrayList 和 LinkedList 的平均 / 最坏时间复杂度都是 $O(n)$** ，都属于线性复杂度。
- 但两者的性能瓶颈来源完全不同：
 - ArrayList：瓶颈在插入位置之后的元素移位；
 - LinkedList：瓶颈在遍历到插入位置 内存开销更大，要存指针

ArrayList vs LinkedList complexities: summary

Method	ArrayList	LinkedList
get(index)	$O(1)$	$O(n)$
add(element)	usually $O(1)$ (very) rarely $O(n)$	always $O(1)$
add(index, element)	$O(n)$	$O(n)$
remove(...)	always $O(n)$	$O(n)$ for general element/index faster if element is near the head / index is small
contains(element)	$O(n)$	$O(n)$

Doubly-linked lists

双向链表是在单链表基础上的升级：

- 每个节点不仅存后继指针 **next**，还存前驱指针 **previous**；

- 节点可以同时知道“我前面是谁”和“我后面是谁”，支持双向遍历。

DoublyLinkedList<E>:

Node<E> head; // 链表头节点

Node<E> tail; // 链表尾节点

int size; // 元素个数

Node<E>:

E data; // 存储元素数据

Node<E> next; // 指向后继节点（下一个）

Node<E> previous; // 指向前驱节点（上一个）

和单链表相比，节点多了一个 **previous** 指针；

链表同时维护 **head** 和 **tail**，方便快速访问两端

Doubly-linked list example: get()

```

Procedure GET(list, k)
if  $k < list.size / 2$  then
     $i \leftarrow 0$ , current  $\leftarrow$  list.head;
    while  $i < k$  do
        current  $\leftarrow$  current.next;
else
     $i \leftarrow list.size - 1$ , current  $\leftarrow$  list.tail;
    while  $i > k$  do
        current  $\leftarrow$  current.previous;
return current.data;
  
```

- Time complexity is $O(m)$, where $m = \min\{k, n - k\}$.
- Similar for insert() operation.
- Can also have separate addFirst() and addLast() operations.

Stacks revisited: model

Definition
A **stack** is a linear (list-based) data structure in which elements may be inserted or removed from only one end (= the *top*).

- Data model is array-based:
Stack<E>:
E[] elements;
int size;
- Can also use a doubly-linked list model, but arrays are faster (hardware).
- Operations: push(element), pop(), peek(), size().
 - No iteration or searching!

:
5
42
-12
2
10

Stacks revisited: algorithms

Procedure PUSH(stack, element)
CHECKCAPACITY(stack);
stack.elements[stack.size] ← element;
stack.size ← stack.size + 1;

Procedure PEEK(stack, element)
if stack.size = 0 then
| **FAIL**; ▷ empty stacks contain no elements
return stack.elements[stack.size];

Procedure POP(stack, element)
if stack.size = 0 then
| **FAIL**; ▷ cannot pop an empty stack
stack.size ← stack.size - 1;
temp ← stack.elements[stack.size];
stack.elements[stack.size] ← null;
return temp;

- In fact, POP() can also check the capacity to avoid excessive waste.
- All complexities are $O(1)$ (constant) except when needing to resize.

主流存储模型：数组实现，备选方案：双向链表实现；但数组依托硬件缓存，访问速度更快。⚠ 栈 ADT 不提供全量遍历、内部查找操作，是栈的约束特性。

Queues revisited

Definition
A **queue** is a linear data structure in which elements are inserted only at one end (the *tail*), and removed only from the other end (the *head*).

Two standard data models:

Array-based; LinkedList-based.
Queue<E>: Queue<E>:
E[] elements; Node<E> head;
int head; Node<E> tail;
int tail; int size;
Node<E>:
E data;
Node<E> next;

Operations:
enqueue(element),
dequeue(), peek(),
size().

- Always constant time/space, except when resizing.
- See Exercise sheet for details.

LinkedList.contains(Object obj)	$O(n)$
LinkedList.remove(int index)	$O(n)$
ArrayList.remove(Object obj)	$O(n)$
LinkedList.remove(Object obj)	$O(n)$
ArrayList.remove(int index)	$O(n)$
ArrayList.contains(Object obj)	$O(n)$
LinkedList.removeFirst()	$O(1)$

各 ADT 选用对应主流存储的底层原因（硬件+算法双重逻辑）

一、栈 Stack → 主流：普通数组

选用理由

1. 操作只在数组尾部（栈顶）增删：push/pop 数组尾部操作天然 $O(1)$ ，不需要挪动元素；
2. 连续内存：CPU 缓存命中率高，硬件读写速度远超链表；
3. 仅需数组+size 变量，内存开销极小，无指针冗余。

备选双向链表：只有需要无限容量、极少扩容场景才用，日常性能吃亏。

二、队列 Queue → 主流：循环数组(circular array)

选用理由

1. 依靠 `head/tail` 下标+取模%实现环形复用空间，头尾增删全 $O(1)$;
2. 连续内存缓存优势，对比链表省去每个节点 `next` 指针的额外内存;
3. 循环数组解决普通数组假溢出问题，空间利用率高。

备选链表：不用预分配容量，但碎片化内存、指针占用额外空间，硬件访问慢。

三、线性表 List → 主流：动态数组 ArrayList; 备选双向链表 LinkedList

动态数组成为主流原因

1. 随机访问 `arr[i] = O(1)`：依靠内存偏移直接寻址，链表随机访问必须遍历 $O(N)$;
2. 尾部新增/删除 $O(1)$ ，绝大多数业务读写集中在尾部;
3. 连续内存缓存友好，批量遍历速度碾压链表。

双向链表仅做备选：

只在高频中间位置插入删除场景使用，其余场景全不如数组。

四、优先队列 PriorityQueue → 主流：堆(数组实现二叉堆)

1. 堆基于数组存储，依靠下标换算父子节点 `left=2*i+1`，无指针开销;
2. 入队、出队（堆顶弹出）稳定 $O(\log N)$ ，综合效率优于二叉搜索树;
3. 连续内存遍历效率更高。

五、Set/Map → 主流：哈希表

1. 哈希映射理论查找/插入平均 $O(1)$ ，远快于平衡树 $O(\log N)$;
2. 绝大多数业务以键值快速查找为核心需求，哈希天生适配;

备选平衡树：仅需要有序遍历、范围查找场景选用。

通用底层规律（一句话总结）💡

主流存储 = 贴合该 ADT 最频繁的操作 + 利用 CPU 连续内存缓存优势，平均时空开销最优；备选是为适配特殊使用场景。

L4 Recursion

Recursive version

Previous procedure is fine, but:

- Need to be careful on bounds of loop;
- Doesn't quite capture *essence* of factorial, which is given by the recurrence relation:

$$n! = n \times (n - 1)!$$

This gives:

Algorithm 2: Recursive factorial

Procedure FACTORIAL(n)

if $n \leq 1$ **then**

return 1;

return $n * \text{FACTORIAL}(n - 1)$;

Recursive algorithms

A recursive algorithm is an algorithm that calls itself.

递归调用通常作用于更小的输入规模上，逐步逼近终止条件。

终止必备要素：

基准情形 (Base Case) / 终止规则：明确算法何时停止递归，避免无限循环。

递归步骤 (Recursive Step)：算法调用自身的核心逻辑，是实现问题分解的关键。

Successive recursive calls are stored in memory on the call stack 连续递归调用会被存储在内存的调用栈中

Space complexity = sum of space complexities of each recursive call

递归就是借助小问题解决大问题

Example 1: factorial

- Recursive step? Easy: if Recursion God can give me $\text{FACTORIAL}(n-1)$, then I can calculate $\text{FACTORIAL}(n) (= n \times \text{FACTORIAL}(n-1))$
- Base case: when does this stop working? When $n = 0$, since $0! = 1 \neq 0 \times (-1)!$
 - Actually, can stop when $n = 1$, as in Algorithm 2.

Algorithm 2: Recursive factorial

Procedure $\text{FACTORIAL}(n)$

if $n \leq 1$ then

 return 1;

▷ base case

return $n * \text{FACTORIAL}(n-1)$;

▷ recursive step

每一行代表一次递归调用，在得到 1! 的结果前，所有调用都处于“等待”状态。

这些等待的调用会被存储在内存的 ** 调用栈 (call stack) ** 中，直到触发基准情形后再逐层回溯计算。

空间复杂度分析

- 单次调用空间：每次递归调用只需要存储少量变量（如参数 n 、返回地址等），占用常数级空间 $O(1)$ 。
 - 调用次数：从 n 到 1，共产生 $O(n)$ 次递归调用，对应调用栈深度为 n 。
 - 总空间复杂度： $O(n) \times O(1) = O(n)$ 。——递归调用需要依赖调用栈来保存中间状态
- 用 for 迭代空间复杂度 $O(1)$

Example 2: Fibonacci sequence

- The famous Fibonacci sequence is given by:
 - $\text{Fib}(0) = 0, \text{Fib}(1) = 1;$
 - $\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$ if $n > 1$.
- Easy enough to implement:

Algorithm 3: naive Fibonacci implementation

Procedure NAIVEFIB(n)**if** $n \leq 1$ **then** **return** n ;**return** NAIVEFIB($n - 1$) + NAIVEFIB($n - 2$);

- However...
 run:
 Fib(50) = 12586269025
 BUILD SUCCESSFUL (total time: 38 seconds)
 ~40 seconds to run on $n=50$!!!

这个实现虽然代码简洁，但存在严重的重复计算问题

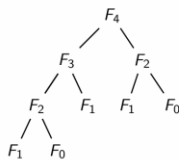
计算 $\text{Fib}(n)$ 时，会重复计算 $\text{Fib}(n-1)$ 和 $\text{Fib}(n-2)$ ，而它们内部又会重复计算更小的子问题。

时间复杂度： $O(2^n)$ （指数级），这也是为什么 $\text{Fib}(50)$ 需要约 40 秒才能运行完成。

空间复杂度： $O(n)$

Another representation: recursion tree

- We can also represent the recursive calls of a recursive algorithm through its **recursion tree**.
- If we write F_k for each call to NAIVEFIB(k), the following shows the recursion tree for NAIVEFIB(4).



If we write $a_k(n)$ for the number of calls to NAIVEFIB(k) in the execution of NAIVEFIB(n), we see that:

- $a_k(n) = \text{Fib}(n+1-k)$ for $k \in [1, n]$
- In particular: $a_1(n) = \text{Fib}(n)$

Formal analysis: time complexity

Algorithm 3: naive Fibonacci implementation**Procedure** NAIVEFIB(n)**if** $n \leq 1$ **then** **return** n ;**return** NAIVEFIB($n - 1$) + NAIVEFIB($n - 2$);

- Clearly, we have: $T(n) = T(n-1) + T(n-2) + c$
 - The constant c encompasses check ($n \leq 1$), addition, return, etc.
- Can solve explicitly; we get:

$$T(n) \sim C \cdot \phi^n$$

- where $\phi := \frac{1 + \sqrt{5}}{2} \approx 1.618 \dots$ is the *golden ratio*.
- Therefore, **exponential** complexity: SLOW!

Improved Fibonacci: FIBLIST

Let's return to our Fibonacci problem.

- The issue was **repeated calculations**: $\text{Fib}(n)$ calculated in $\text{Fib}(n+2)$ and $\text{Fib}(n+1)$.
 - We should only need to calculate it once, then “remember” the value.
- Idea: use a list!

Algorithm 4: calculating first n terms of Fibonacci sequence

Procedure FIBLIST(n)

if $n = 0$ **then**

return [0];

if $n = 1$ **then**

return [0, 1];

list $\leftarrow$ FIBLIST($n - 1$);

nextTerm $\leftarrow$ list.get($n - 1$) + list.get($n - 2$);

list.add(nextTerm);

return list;

优化：输出包含 0- n 的列表，只计算每个 $\text{Fib}(k)$ 一次，然后将结果存储在列表中，后续直接读取，避免重复计算

时间空间复杂度 $O(n)$

Improved Fibonacci: FASTFIB

Using this, calculating $\text{Fib}(n)$ is easy!

Algorithm 5: a fast Fibonacci algorithm

Procedure FASTFIB(n)

fibList $\leftarrow$ FIBLIST(n);

return fibList.get(n);

- Notice that the FASTFIB procedure is not itself recursive.
- Instead, the recursive step is in an auxiliary sub-procedure FIBLIST.
 - We refer to FIBLIST as a *helper* procedure / algorithm.
- The use of helper algorithms to solve problems recursively is a common strategy to optimise their time and space complexities.
 - More on this shortly.
- The complexity of FASTFIB is just the complexity of FIBLIST, so let's analyse that.

将“生成完整序列”的复杂递归逻辑封装在辅助函数 FIBLIST 中，主函数只做简单的“调用 + 取值”，让代码更清晰、易维护。

复用性：FIBLIST 可以被其他需要完整斐波那契序列的函数复用，避免重复代码。

复杂度继承：FASTFIB 的时间和空间复杂度完全由 FIBLIST 决定，即：

- 时间复杂度： $O(n)$

- 空间复杂度：O(n)

Yet another Fibonacci	ITERFIB: an iterative Fibonacci algorithm
Key idea Storing all values $\text{Fib}(0), \dots, \text{Fib}(n)$ is unnecessary. In fact, we only need $\text{Fib}(n)$ and $\text{Fib}(n-1)$. Algorithm 6: space- and time-efficient recursive Fibonacci Procedure LASTTWO$\text{Fib}(n)$ Output: The last two values $\text{Fib}(n)$ and $\text{Fib}(n-1)$ in the Fibonacci sequence <pre> if $n = 1$ then return [0, 1]; prev ← LASTTWO$\text{Fib}(n-1)$; nextTerm ← prev[0] + prev[1]; return [prev[1], nextTerm]; </pre> Procedure EFFICIENT$\text{Fib}(n)$ <pre> if $n = 0$ then return 0; return LASTTWO$\text{Fib}(n)$[1]; </pre>	Algorithm 7: an iterative Fibonacci algorithm Procedure ITER$\text{Fib}(n)$ <pre> if $n = 0$ then return 0; previous ← 0, current ← 1; for $i \leftarrow 2$ to n do temp ← current; current ← previous + current; previous ← temp; return current; </pre> • Time complexity: linear $O(n)$ (one loop with constant operations); • Space complexity: constant $O(1)$.

迭代 vs 递归：优缺点对比

理论等价性

- **递归 → 迭代：**任何递归算法都可以转换为迭代版本（Böhm-Jacopini 定理证明：所有程序都可仅用条件语句和循环表达）。
- **迭代 → 递归（部分成立）：**反之亦然，尤其在函数式编程语言中，通常只允许递归而不提供循环语法。

递归算法的优缺点

✅ 优点：

- 代码更优雅、简洁，能精准捕捉问题的“本质”，符合数学定义的直觉（如阶乘、斐波那契）。
- 适合处理分治、回溯等天然具有递归结构的问题。

❌ 缺点：

- naive 实现可能极慢（如指数级斐波那契）。
- 依赖调用栈保存中间状态，内存开销更大，存在栈溢出风险。
- 空间复杂度通常为 $O(n)$ （递归深度），高于迭代版的 $O(1)$ 。

迭代算法的优缺点

✅ 优点：

- 空间效率更高（通常 $O(1)$ ），无栈溢出风险。
- 执行速度通常更快，避免了函数调用的额外开销。

- 更可控，适合性能敏感的场景。

✖ 缺点：

- 代码往往更繁琐，需要手动管理状态（如循环变量、临时变量），可读性不如递归。
- 对于复杂递归结构（如树遍历），实现起来更复杂。

减治法（Decrease-and-Conquer）

1. 最常见形式：每次减 1

递推关系：

$$T(n) = T(n - 1) + O(1)$$

- 含义：问题规模每次减 1，额外开销是常数时间。
- 求解：展开后 $T(n) = T(0) + \sum_{i=1}^n O(1) = O(n)$ 。
- 例子：递归阶乘、递归线性查找、EFFICIENTIB 等。
- 结论：线性时间复杂度 $O(n)$ 。

2. 更一般形式：额外开销为多项式

递推关系：

$$T(n) = T(n - 1) + O(n^k)$$

- 含义：问题规模每次减 1，额外开销是 $O(n^k)$ 级别的多项式。
- 求解：
$$T(n) = \sum_{i=1}^n O(i^k) = O\left(\int_1^n x^k dx\right) = O(n^{k+1})$$
- 例子：若 $k = 1$ ，则 $T(n) = O(n^2)$ ；若 $k = 2$ ，则 $T(n) = O(n^3)$ 。
- 结论：时间复杂度为 $O(n^{k+1})$ 。

3. 减治法的定义

更广义的减治法满足：

$$T(n) = 1 \cdot T(\alpha_n) + f(n)$$

其中：

- $\alpha_n < n$ ：问题规模每次严格减小；
- $f(n)$ ：本次递归的额外开销；
- 只有一个子问题（区别于分治法的多个子问题）。

特殊情况：

若 $\alpha_n = n - k$ (k 是固定常数，比如 $k = 1$)，称为按常量递减（decrease by a constant）。

L5 Searching, sorting

查找算法

线性查找

- 迭代版：遍历数组每个元素，找到目标返回 true，遍历完未找到返回 false 时间复杂度 $O(n)$ ，空间复杂度 $O(1)$
- 递归版：借助辅助函数，从起始索引开始递归查找，直到数组末尾时间复杂度 $O(n)$ ，空间复杂度 $O(n)$ （递归调用栈）

Reminder: linear search

Recall from previous weeks:

Algorithm 1: iterative linear search

```
Procedure ITERATIVELINEARSEARCH(arr, val)
foreach item in arr do
    if item = val then
        return true;
return false;
```

- Simple iterative version.
- Linear time complexity $O(n)$, constant space complexity $O(1)$.

Recursive linear search

Last week, we also saw:

Algorithm 2: recursive linear search

```
Procedure LINEARSEARCHHELPER(arr, val, startIndex)
Output: true if arr contains val at an index  $\geq$  startIndex, false otherwise
if startIndex = arr.length then
    return false;                                ▷ base case
if arr[startIndex] = val then
    return true;
return LINEARSEARCHHELPER(arr, val, startIndex + 1);
```

```
Procedure RECURSIVELINEARSEARCH(arr, val)
return LINEARSEARCHHELPER(arr, val, 0);
```

Linear time and space complexities.

二分查找

- 前提：数组已排序
- 核心思想：每次取中间元素，与目标比较，排除一半数据，分治法思路
- 递归版：用辅助函数记录起始、结束索引，中间值匹配则返回，否则递归查找左 / 右半区
- 迭代版：用循环替代递归，更新起始 / 结束索引，空间复杂度优化为 $O(1)$
- 复杂度：时间 $O(\log n)$ ，递归版空间 $O(\log n)$ ，迭代版 $O(1)$

The binary search algorithm: iterative version

Recall that any recursive algorithm has an iterative transcription.

- Generally, iterative version is (schematically):
while (base case not reached)
repeat recursive step

```
Procedure ITERATIVEBINARYSEARCH(arr, val)
startIndex ← 0, endIndex ← arr.length;
while startIndex < endIndex do
  midIndex ← (startIndex + endIndex) / 2;
  if arr[midIndex] = val then
    return true;
  else if arr[midIndex] < val then
    startIndex ← midIndex + 1;
  else
    endIndex ← midIndex;
return false;
```

The binary search algorithm: recursive version

As in RECURSIVELINEARSEARCH, to avoid extracting sub-arrays, we use a helper algorithm.

- Helper takes two additional parameters: startIndex and endIndex.

```
Procedure BINARYSEARCHHELPER(arr, val, startIndex, endIndex)
Output: true if there exists an index i with startIndex ≤ i < endIndex such that
arr[i] = val, false otherwise
if startIndex = endIndex then
  return false;
midIndex ← (startIndex + endIndex) / 2;
if arr[midIndex] = val then
  return true;
else if arr[midIndex] < val then
  return BINARYSEARCHHELPER(arr, val, midIndex + 1, endIndex);
else
  return BINARYSEARCHHELPER(arr, val, startIndex, midIndex);
Procedure BINARYSEARCH(arr, val)
return BINARYSEARCHHELPER(arr, val, 0, arr.length);
```

Searching sorted arrays

- Suppose that we have an array arr of numbers, which are indexed in increasing order.
 - That is: $\text{arr}[0] < \text{arr}[1] < \dots < \text{arr}[n-1]$
- We are searching for a value val.
- Fix an index m.
 - If $\text{arr}[m] = \text{val}$, we're done.
 - If $\text{arr}[m] < \text{val}$, then by the sorted property val can only be found in $\text{arr}[m+1:n]$.
 - Similarly, if $\text{arr}[m] > \text{val}$, then val can only be found in $\text{arr}[0:m]$.
- If we take $m = \frac{n}{2}$, this allows us to "throw away" half of the array.
 - This is the idea of binary search, and on divide-and-conquer algorithms in general.

N.B.: In the above, the notation $\text{arr}[a:b]$ is short-hand for the sub-array given by $[\text{arr}[a], \text{arr}[a+1], \dots, \text{arr}[b-1]]$

Complexity analysis: BINARYSEARCH ($2/2$)

- We have: $T(n) = T\left(\frac{n}{2}\right) + O(1)$
- What is $T(n)$?
- Since each recursive call adds constant time, $T(n)$ is essentially the number of recursive calls.
 - In each recursive call, input size is divided by 2.
 - After one recursive call, input size is $n/2$.
 - After two recursive calls, input size is $(n/2)/2 = n/4$.
 - After k recursive calls, input size is $n/2^k$.
- The total number of recursive calls is therefore the value of k such that $n/2^k \approx 1$.
 - This is in fact just $k = \log_2(n)$
- Final analysis: $T(n) = O(\log(n))$
- Space complexity is the size of the call stack, which again is the number of recursive calls, so $M(n) = O(\log(n))$

排序算法

序关系定义

集合上的二元关系，满足三分性（两元素必满足小于、等于、大于其一）与传递性（ $a < b$ 且 $b < c$ 则 $a < c$ ），是排序的基础。

Order relation: definition

Given a set A , a **binary relation** on A is a mapping:

$\mathcal{R} : A \times A \rightarrow \{\mathbf{true}, \mathbf{false}\}$. We write $a_1 \mathcal{R} a_2$ when $\mathcal{R}(a_1, a_2) = \mathbf{true}$.

Definition

A (total) **order relation** on a set A is a binary relation $<$ satisfying the following two conditions:

- *Trichotomy*: For any pair $(a, b) \in A \times A$, exactly one of $a < b$, $a = b$, $b < a$ holds.
- *Transitivity*: For any triple (a, b, c) , if $a < b$ and $b < c$, then we have $a < c$.

We say that $(A, <)$ is an *ordered set*.

可排序对象

数字、字母（字典序）、学生信息（学号、姓名等）；复数无兼容乘法的序关系。

字典序

按位依次比较的序列排序规则，如单词字典序、多字段组合排序（姓名 + 年级 + 学号）。

Lexicographical order: definition

Suppose we have ordered sets $(A_1, <_1), (A_2, <_2), \dots$, and we want to order sequences $(a_1, a_2, \dots) \in A_1 \times A_2 \times \dots$

- Classic example: we know how to order *letters*, how do we order *words* (sequences of letters)?

Definition

The **lexicographical order** $<_{\text{lex}}$ on $A_1 \times A_2 \times \dots$ is defined as follows. For two sequences $a := (a_1, a_2, \dots)$, $a' := (a'_1, a'_2, \dots)$, we say that $a <_{\text{lex}} a'$ if there exists $i \geq 1$ such that $a_i <_i a'_i$ and $a_j <_j a'_j$ for all $j < i$.

How does the lexicographical order work?

- Compare a_1 and a'_1 . If $a_1 < a'_1$ then $a < a'$; if $a'_1 < a_1$ then $a' < a$.
- If $a_1 = a'_1$, compare a_2 and a'_2 as above, then repeat.

排序算法定义

Definition

A **general sorting algorithm** is an algorithm that sorts an input array using only pairwise *comparisons* of elements and positional *swaps*.

Our general framework therefore assumes:

- Input: an array whose elements are members of a given ordered set $(A, <)$; for example, an array of numbers.
- Two elementary operations:
 - $\text{LESS}(a, b)$, which returns **true** if $a < b$, and **false** otherwise;
 - $\text{SWAP}(\text{arr}, i, j)$, which swaps the elements of arr at indices i and j , leaving other elements unchanged.

Both operations are assumed to run in constant $O(1)$ time, irrespective of their inputs.

Bubble sort

核心：多轮遍历，相邻元素逆序则交换，大元素逐步“冒泡”到末尾

优化：某轮无交换则数组已有序，直接终止

复杂度：最坏 / 平均 $O(n^2)$ ，最好 $O(n)$ ，空间 $O(1)$

Bubble sort: implementation

- Key observation: after first pass, the largest element of the array will be moved to its correct position at the end of the array.
- As such, next pass can stop one position earlier, and so on.

This gives us the following pseudocode implementation:

Algorithm 3: Bubble sort

Procedure BUBBLESORT(arr)

```
 $n \leftarrow \text{arr.length}; \text{swapsNeeded} \leftarrow \text{false}; k \leftarrow n - 1;$ 
▷  $k$  is the index to which the largest remaining element “bubbles up” to on each pass
while  $k > 0$  do
  for  $i \leftarrow 0$  to  $k - 1$  do
    if  $\text{LESS}(\text{arr}[i], \text{arr}[i+1])$  then
       $\text{SWAP}(\text{arr}, i, i+1);$ 
       $\text{swapsNeeded} \leftarrow \text{true};$ 
  if  $\text{!swapsNeeded}$  then
    return ; ▷ if no swaps needed on a whole pass, the array is sorted
```

Bubble sort: complexity analysis

Time complexity

- Sufficient to count comparisons, since $0 \leq \# \text{swaps} \leq \# \text{comparisons}$.
- Inner loop always has k comparisons.
- In worst case, where the original array is in reverse order, we can see that swaps are needed on each pass, so all passes are necessary.

- Worst-case number of comparisons is therefore $\sum_{k=1}^{n-1} k = \frac{n(n-1)}{2}$.

- This gives: $T_{\text{worst}}(n) = O(n^2)$

- Average case (complicated) also gives $T_{\text{avg}}(n) = O(n^2)$
- Best case: array is already sorted, first pass detects that no swaps are needed, giving $T_{\text{best}}(n) = O(n)$

Space complexity

- No additional memory required, since algorithm is **in-place**, so space complexity is $M(n) = O(1)$

Insertion sort

核心：将数组分为已排序 / 未排序区，依次把未排序首元素插入已排序区正确位置

特点：符合人工整理数据的直觉，原地排序

复杂度：最坏 / 平均 $O(n^2)$ ，最好 $O(n)$ ，空间 $O(1)$

Insertion sort: implementation

Algorithm 4: Insertion sort

```
Procedure INSERTIONSORT(arr)
for  $i \leftarrow 1$  to  $arr.length - 1$  do
     $j \leftarrow i$ ; ▷ insertion index
    while  $j > 0$  and  $LESS(arr[j], arr[j-1])$  do
        SWAP( $j, j-1$ );
         $j \leftarrow j-1$ ;
```

- Standard implementation uses a shifting mechanism (see Week 3 INSERT(list, item) procedure) rather than swaps.
 - Faster, as a swap is at least two assignments (three in most languages).
- But our chosen model allows only swaps and comparisons, so we present a swap-based version.

Insertion sort: complexity analysis

Time complexity

- If we denote $R(i)$ for the number of repetitions of the inner loop, then the complexity is $\sim \sum_{i=1}^{n-1} R(i)$.
- Worst-case: array is in reverse order, each successive element bubbles all the way to the front of the array, giving $R(i) = i$. Complexity is again $T(n) = O(n^2)$ (quadratic).
- Average-case: on average, insertions are in the middle of the sorted portion, so $R(i) \approx \frac{i}{2}$. Improves only by constant, still quadratic $O(n^2)$ complexity.
- Best-case: if the array is sorted, no swaps are needed, i.e. $R(i) = 1$, so that $T(n) = O(n)$ (linear).

Space complexity

- In-place sorting: $M(n) = O(1)$.

Selection sort

核心：每轮找未排序区最小值，交换到未排序区首位

特点：交换次数固定为 $n-1$ 次，适合交换成本高的场景

复杂度：最好 / 最坏 / 平均均为 $O(n^2)$ ，空间 $O(1)$

Selection sort: implementation

Algorithm 5: Selection sort

```
 $n \leftarrow arr.length$ ;
for  $i \leftarrow 0$  to  $n - 2$  do
     $minIndex \leftarrow i$ ;  $currentMin \leftarrow arr[i]$ ;
    for  $j \leftarrow i + 1$  to  $n - 1$  do
        if  $LESS(arr[j], currentMin)$  then
             $minIndex \leftarrow j$ ;  $currentMin \leftarrow arr[j]$ ;
    SWAP( $i, minIndex$ );
```

- Loops are fixed size for-loops, so the time complexity will be the same in best, worst and average cases.
- Outer loop has $\approx n$ iterations, inner loop has $\approx n - i$ iterations, giving $T(n) = O(n^2)$ (quadratic).
- However, the complexity is borne by the *comparisons*, since the number of *swaps* is $n - 1 = O(n)$.
 - If swaps are much more costly than comparisons, this makes selection sort a better choice than other algorithms seen so far.
- Space complexity is still $M(n) = O(1)$ (constant, since in-place).

L6 Faster sorting

分治法 (Divide-and-Conquer)

核心思想：把数组分成两半→分别递归排序→合并结果，递推式： $T(n)=2T(n/2)+O(n)$ 。

归并排序 (Merge Sort)

步骤

- 分 (Divide)：将数组拆分为左右等长子数组
- 治 (Conquer)：递归排序子数组
- 合：合并两个有序子数组 merge the two sorted halves into a single sorted array

实现

- 递归实现，需辅助数组，非原地排序 is not in-place.

复杂度

- 时间： $\Theta(n \log n)$ （最好 / 平均 / 最坏相同） $T(n) = 2 \cdot T(n/2) + O(n)$
- 空间： $\Theta(n)$ （辅助数组）

特性：稳定排序

Merge sort: implementation

Algorithm 3: merge sort

```

Procedure MERGESORT( $T[]$  arr)
  temp  $\leftarrow$  new  $T$ [arr.length];
  MERGESORTHELPER(arr, temp, 0, arr.length);

Procedure MERGESORTHELPER(arr, temp, start, end)
  if end - start  $\leq 1$  then
    return ;  $\triangleright$  If the sub-array has length at most 1, nothing to do
  mid  $\leftarrow$  (start + end) / 2;
  MERGESORTHELPER(arr, temp, start, mid);
  MERGESORTHELPER(arr, temp, mid, end);
  MERGE(arr, temp, start, mid, end);

```

- Obviously incomplete: need to implement MERGE procedure.
- Still useful: we've *decomposed* the problem, and *reduced* it to the MERGE problem.

The merge operation: implementation

Algorithm 4: merging two sorted arrays

```

Input: An array arr and indices start, mid, end such that arr[start:mid] and arr[mid:end] are sorted
Output: The modified array arr such that arr[start:end] is sorted
Copy(arr, temp, start, end);  $\triangleright$  copy the sub-array arr[start:end] into temp
 $i \leftarrow$  start,  $j \leftarrow$  mid,  $k \leftarrow$  start;
 $\triangleright$  indices for left sub-array, right sub-array, merged result
while  $i < mid$  and  $j < end$  do
  if LESS(temp[j], temp[i]) then
    arr[k]  $\leftarrow$  temp[j],  $j \leftarrow j + 1$ ;
  else
    arr[k]  $\leftarrow$  temp[i],  $i \leftarrow i + 1$ ;
  k  $\leftarrow$  k + 1;

```

```

while  $i < mid$  do
  arr[k]  $\leftarrow$  temp[i],  $i \leftarrow i + 1$ ,  $k \leftarrow k + 1$ ;
 $\triangleright$  copy remaining elements from left-half, if any
 $\triangleright$  no need to copy remaining elements from right-half, as already in correct place

```

This implementation ensures that merge sort is **stable**.

快速排序（Quick Sort）

1. 步骤

- 选基准（pivot）
- 分区：小于基准放左，大于放右
- 递归排序左右子数组

2. 实现

- 递归实现，原地排序

3. 复杂度

- 最好 / 平均： $O(n \log n)$
- 最坏： $O(n^2)$ （基准选择极差）
- 空间：平均 $O(\log n)$ ，最坏 $O(n)$

4. 特性：不稳定排序，实际运行最快

The partition step: implementation

In fact, various implementations exist. Here is an example.

Algorithm 6: the partition step in quick sort

```

Procedure PARTITION(arr, start, end, pivotIndex)
  pivot ← arr[pivotIndex], i ← start;
  SWAP(arr, pivotIndex, end - 1); ▷ moves the pivot out of the way
  for j ← start to end - 2 do
    if LESS(arr[j], pivot) then
      SWAP(arr, i, j);
      i ← i + 1;
  SWAP(arr, i, end - 1); ▷ put the pivot back in the right place
  return i;
  
```

- Partitions the sub-array arr[start:end] around the pivot, and returns the pivot's index in the partitioned array.
- Standard implementation uses " $\leq$ " for the comparison.

Quick sort: complexity analysis (2/2)

Time complexity – summary

- For poor pivot choices, we get $T_{\text{worst}}(n) = O(n^2)$
- For good pivot choices, we get $T_{\text{best}}(n) = T_{\text{avg}}(n) = O(n \log(n))$
- In general, quick sort is quicker than merge sort due to smaller multiplicative constants and lower memory overhead.

Space complexity

- Quick sort is **in place**: no need for an auxiliary array.
- Memory complexity is depth of the recursion tree. Again, depends on having good pivot choices.
 - For poor choices, we get $M_{\text{worst}}(n) = O(n)$
 - For good choices, we get $M_{\text{best}}(n) = M_{\text{avg}}(n) = O(\log(n))$

原地和稳定

稳定：原数组中值相等的两个元素，排序结束后，相对先后顺序保持不变。

原地排序：排序全程仅使用常数级额外空间 $O(1)$ ，不新建和输入长度 N 成正比的辅助数组，仅在原输入数组

	排序算法	是否原地
2	冒泡排序	✓ 原地 $O(1)$
3	插入排序	✓ 原地 $O(1)$
4	选择排序	✓ 原地 $O(1)$
5	希尔排序	✓ 原地 $O(1)$
5	快速排序	✓ 原地
7	堆排序	✓ 原地
3	归并排序	✗ 非原地 $O(N)$

点击图片可查看完整电子表格

复杂度分析工具 lower bounds and Big-Theta

1. 渐进符号

- $O(g(n))$: 上界
- $\Omega(g(n))$: 下界

- $\Theta(g(n))$: 确界 (同时满足 O 与 Ω)

Definition (lower bound)

We say that $f(n)$ is a **Big-Omega** of $g(n)$, and write $f(n) = \Omega(g(n))$ if $g(n) = O(f(n))$, equivalently $f(n) \geq c \cdot g(n)$.

We say that $f(n)$ is a **Big-Theta** of $g(n)$, and write $f(n) = \Theta(g(n))$ if $f(n)$ is both a Big-Oh and a Big-Omega of $g(n)$, equivalently $c \cdot g(n) \leq f(n) \leq C \cdot g(n)$.

定义 (下界)

- 我们称 $f(n)$ 是 $g(n)$ 的 **Big-Omega (大欧米伽)**, 记作 $f(n) = \Omega(g(n))$, 当且仅当 $g(n) = O(f(n))$, 等价定义为: $f(n) \geq c \cdot g(n)$
- 我们称 $f(n)$ 是 $g(n)$ 的 **Big-Theta (大西塔)**, 记作 $f(n) = \Theta(g(n))$, 当且仅当 $f(n)$ 同时是 $g(n)$ 的 $O(g(n))$ 和 $\Omega(g(n))$, 等价定义为: $c \cdot g(n) \leq f(n) \leq C \cdot g(n)$

2. 主定理

- 公式: $T(n) = aT(n/b) + f(n)$
- 临界指数: $c_{\text{crit}} = \log_b(a)$
- 归并排序对应 Case 2, 得 $\Theta(n \log n)$

Master theorem

Suppose that we have a recurrence relation of the form

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

We define the critical exponent $c_{\text{crit}} := \log_b(a) = \frac{\log(a)}{\log(b)}$.

Case 1 If $f(n) = O(n^c)$ for some $c < c_{\text{crit}}$, then we have

$$T(n) = \Theta(n^{c_{\text{crit}}})$$

Case 2 If $f(n) = \Theta(n^{c_{\text{crit}}} \cdot (\log(n))^k)$ for some $k \geq 0$, then we have

$$T(n) = \Theta(n^{c_{\text{crit}}} \cdot (\log(n))^{k+1})$$

Case 3 If $f(n) = \Omega(n^c)$ for some $c > c_{\text{crit}}$, and some additional "regularity" condition holds on f , then we have

$$T(n) = \Theta(f(n))$$

主定理的使用步骤 (考试 / 刷题秒用)

1. 提取参数：从递推式中找到 $a, b, f(n)$
2. 计算临界指数： $ccrit = \log_b a$
3. 对比 $f(n)$ 和 n^{ccrit} ：
 - 若 $f(n)$ 比 n^{ccrit} 慢 $\rightarrow$ 用情况 1
 - 若 $f(n)$ 和 n^{ccrit} 同量级（带对数项） $\rightarrow$ 用情况 2
 - 若 $f(n)$ 比 n^{ccrit} 快且满足正则条件 $\rightarrow$ 用情况 3

非比较类排序

1. 计数排序（Counting Sort）依托数值范围开辟计数数组统计频次，再回填原数组
 - 假设：元素为 $0 \sim m-1$ 整数
 - 时间： $O(n+m)$
 - 空间： $O(m)$
 - 稳定、非原地

Counting sort: implementation

Algorithm 7: counting sort
Input: An array with values in the range $\{0, \dots, m-1\}$
Output: The sorted array
Procedure COUNTINGSORT(arr)
 $count \leftarrow \text{new int}[m];$ $\triangleright$ auxiliary array, initialised to all 0's
 foreach $item$ **in** arr **do**
 $count[item]++;$ $\triangleright$ count how many times $item$ appears in arr
 $index \leftarrow 0;$
 for $value \leftarrow 0$ **to** $m-1$ **do**
 while $count[value] > 0$ **do**
 $arr[index] \leftarrow value, index++;$
 $count[value]--;$

Counting sort: complexity analysis

Time complexity

- Initialising new array is $O(m)$.
- First loop is simple loop over $arr \Rightarrow O(n)$
- Second loop:
 - Notice that inside the while loop, we always increment the index.
 - At most n such incrementations, so the cumulative number of operations for the while loop is $O(n)$.
 - There are also m iterations of the for loop, giving $O(m)$.

Therefore, total time complexity is $T(n) = O(m+n)$

- Faster than comparison-based sorting algorithms when m is relatively small, e.g. $m = O(n)$.

Space complexity

- Auxiliary array of length m , so space complexity is $M(n) = O(m)$

统计计数：新建 $count[]$ ，遍历原数组， $count[val]++$ 统计每个数字出现次数；

前缀和（保证稳定）： $count[i] += count[i-1]$ ，得到每个值在有序数组里的最后下标；

逆序回填原数组：从后向前遍历原数组，按 $count[val]$ 放到结果位置， $count[val]--$ 。

优点 1. 不做元素比较，线性时间，数据密集、值域小时效率碾压排序； 2. 天然稳定，是基数排序的内层依赖算法。

缺点 1. 只能排整数，无法处理浮点数、字符串； 2. 值域极大时（如 $[1, 1000000]$ ）过大，空间爆炸失效。

2. 基数排序（Radix Sort）非比较、稳定排序；依托计数排序做每一位辅助，从低

位→高位依次排序

- 按**最低位到最高位**稳定排序
- 时间： $O(d(n+b))$ (d 位数, b 基数)
- 空间： $O(n+b)$
- 稳定、非原地

n : 元素个数; d : 数字最大位数; k : 数位取值范围 (0~9, $k=10$)

时间: $O(d \cdot (n+k))$

空间: $O(n+k)$, 非原地

Radix sort: partial implementation

Algorithm 8: radix sort

Procedure RADIXSORT(arr)

Input: An array of non-negative integers, each with at most d digits

Output: The array, sorted

for $i \leftarrow 0$ **to** $d-1$ **do**

 COUNTSORTBYDIGIT(arr, i);

Procedure COUNTSORTBYDIGIT(arr, i)

count $\leftarrow$ **new** int[10];

foreach $item$ **in** arr **do**

 digit $\leftarrow (item / 10^i) \% 10$;

 count[digit]++;

> re-arrange items in arr according to count

- The radix sort algorithm is stable.
- The (hidden) final step requires a copy of the input array arr , so the radix sort algorithm is not in-place.

Radix sort: complexity analysis

Time complexity

- Suppose each item has at most d digits when written in base b (previous slides have $b = 10$).
- There are d calls to COUNTSORTBYDIGIT.
- Each digit has b possible values, so the complexity of COUNTSORTBYDIGIT is $O(n+b)$.

Overall complexity: $T(n) = O(d \cdot (n+b))$

Space complexity

- Final step in COUNTSORTBYDIGIT requires $O(n)$.
- Auxiliary array count has length b .

Overall complexity: $M(n) = O(n+b)$

流程 (LSD 低位优先)

- 取当前数位 (先个位→十位→百位)
- 对该位用**计数排序** (保证稳定)
- 全部位数处理完, 数组整体有序

优点: 线性级效率, 大数据整数排序很快

缺点: 只能整数, 不能小数、无规律大数

Summary: complexities

Algorithm	Time (best)	Time (avg)	Time (worst)	Space
Bubble sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge sort	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$	$O(n)$
Quick sort	$O(n \log(n))$	$O(n \log(n))$	$O(n^2)^*$	$O(\log(n))$
Counting sort	$O(n+m)$	$O(n+m)$	$O(n+m)$	$O(m)$
Radix sort	$O(d \cdot (n+b))$	$O(d \cdot (n+b))$	$O(d \cdot (n+b))$	$O(n+b)$

Summary: use cases

Algorithm	Stable	In-place	When to use
Bubble sort	Y	Y	Almost never
Insertion sort	Y	Y	Small or nearly sorted data
Selection sort	N	Y	When swaps are costly
Merge sort	Y	N	When stability is important, or need $O(n \log(n))$ guarantee
Quick sort	N	Y	General-purpose sorting (fastest on average)
Counting sort	Y	N	Small range of (integer) values
Radix sort	Y	N	Large n , small digit range

L8 Union Find

Union-Find

- 定义：并查集 (Disjoint Set)，用于管理互不相交的集合，支持动态合并与查询。
- 核心操作
 - a. **union(x, y)**: 将 x 和 y 所在集合合并
 - b. **isSameGroup(x, y)**: 判断 x、y 是否同属一个集合
- 应用：Kruskal 最小生成树、计算渗透理论、动态连通维护。

Union Find on Integers

- Let's start simple, we let:
 - the items are integers instead of arbitrary data, and we
 - declare the number of items in advance, where everything is *disconnected*
- We want to implement two methods: (and a constructor)

```
/** Combines two items p and q. */
void union(int p, int q) {
    ...
}

/** Decides if two items p and q are in the same group. */
boolean isSameGroup(int p, int q) {
    ...
}
```

check **find(p) == find(q)**
find(p) → the group id of p

- where the **number of items N** and the **number of operations M** can be very large
- and the two methods are called in *arbitrary* order

1. ListOfSetsUF (朴素实现)

- 用集合列表存储连通分量。
- 每次合并 / 查询都要遍历所有集合，时间复杂度 **O(N)**。

Idea 1: List of Sets

- Using `List<Set<Integer>>` from Java Library
 - A very intuitive idea
 - However, imagine in the beginning, we have this List of Sets of integers:
[`{0}`, `{1}`, `{2}`, `{3}`, `{4}`, `{5}`, `{6}`]
 - We have to iterate through all the sets to find an item
 - complicated and slow!
 - Worst case: if nothing is combined, then `isSameGroup(5, 6)` requires iterating through N-1 sets to find 5, then N sets to find 6
 - overall runtime is linear or $O(N)$
 - similarly for union

2. QuickFind (快速查找)

- 用数组 `id[]`, `id[i]` 表示元素 `i` 所在组号。 `id[i]=1` (1 就是组号的意思)
- **isSameGroup**: 直接比较 `id[p] == id[q] → O(1)`。
- **union**: 遍历数组, 把所有 `p` 的组号改成 `q` 的组号 → **O(N)**。

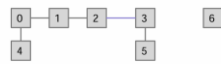
Idea 2: Array of Integers (3)

- Using an array of integers, where the `i`th entry gives the set number/id of item `i`
 - **union**(`p`, `q`) simply changes all entries that equal `id[p]` to `id[q]`

Before union(2, 3) operation:



After union(2, 3) operation:



{0, 1, 2, 4}, {3, 5}, {6}

{0, 1, 2, 4, 3, 5}, {6}

int[] id [4 4 4 5 5 6]
 0 1 2 3 4 5 6

int[] id [5 5 5 5 5 6]
 0 1 2 3 4 5 6

the set number of 0, 1, 2, 4 changes from 4 to 5, which is the set number of 3

Idea 1: List of Sets

- Using `List<Set<Integer>>` from Java Library
 - A very intuitive idea
 - However, imagine in the beginning, we have this List of Sets of integers: `[[{0}, {1}, {2}, {3}, {4}, {5}, {6}]`
 - We have to iterate through all the sets to find an item
 - complicated and slow!
 - Worst case: if nothing is combined, then `isSameGroup(5, 6)` requires iterating through `N-1` sets to find 5, then `N` sets to find 6
 - overall runtime is linear or $O(N)$
 - similarly for union

3. QuickUnion (快速合并)

- 用数组 `parent[]`, `parent[i]` 表示 `i` 的父节点, 根节点为 -1。
- **find**: 向上找根, 最坏树高 **O(N)**。
- **union**: 直接把一个根挂到另一个根下, 可能形成长链, 最坏 **O(N)**。

4. Weighted Quick Union (加权快速合并, 最优)

- 维护 `size[]` 记录每棵树的元素数量。
- 合并规则: 小树的根 接到 大树的根, 避免树过高。
- 树高: 严格控制为 **O(log N)**。

Queue and Binary Heap

优先队列 (Priority Queue)

- 定义: 抽象数据结构, 支持添加元素 (**add**) 与删除最小元素 (**delMin**)。
- 最小优先队列 (MinPQ) 接口:
 - `add(T item)`: 添加元素
 - `getMin()`: 获取最小元素
 - `T delMin()`: 删除并返回最小元素
 - `isEmpty()`: 判断是否为空

- `size()`: 获取元素数量
- 典型应用：查找流中最大 M 个元素、Dijkstra 算法、Prim 算法、Huffman 编码、任务调度、中断处理等。

Application Example

- Problem : Find the *largest* M items in a stream InputStream of N items
 - Fraud detection : isolate transactions
 - File maintenance : find biggest files or directories
- Constraint : Not enough memory to store N items

N huge, M large

```
MinPQ<Transaction> pq = new MinPQ<Transaction>();
while (InputStream.hasNextLine()) {
    String line = InputStream.readLine();
    Transaction item = new Transaction(line);
    pq.add(item);
    if (pq.size() > M)
        pq.delMin();
}
```

// 若堆大小超过 M，删除当前堆中最小的元素

应用

	英文原文	中文翻译
1		
2	Customers in a line, Colliding particles simulations	顾客排队、粒子碰撞模拟
3	Reducing roundoff error in numerical computation	减少数值计算中的舍入误差
4	Huffman codes	哈夫曼编码
5	Dijkstra's algorithm, Prim's algorithm	Dijkstra（迪杰斯特拉）算法、Prim
6	Sum of powers	幂和问题
7	A* search	A* 搜索算法
8	Maintain largest M values in a sequence	在序列中维护最大的 M 个值
9	Load balancing, Interrupt handling	负载均衡、中断处理
0	Bin packing, Scheduling	装箱问题、任务调度
1	Bayesian spam filter	贝叶斯垃圾邮件过滤器

点击图片可查看完整电子表格

MinPQ Interface

- Minimum Priority Queue supports fast removal of the smallest item in the data structure and fast item addition
- To have a notion of smallest, either we implement the item to be comparable, or explicitly specify the priority of an item with a comparable value, for example an integer
- Interface MinPQ with comparable items:

```
public interface MinPQ<T extends Comparable<T>> {  
    void add(T item);  
    T getMin();  
    T delMin();  
    boolean isEmpty();  
    int size();  
}
```

Binary Heaps

朴素实现的性能瓶颈

	实现方式	添加元素 (add)	删除最小元素 (delMin)	核心问题
1	普通数组（无序）	$O(1)$	$O(N)$	找最小值需要遍历整个数组
2	有序数组（预排序）	$O(N)$	$O(1)$	插入元素时需要移动元素来维持有序

点击图片可查看完整电子表格

两种方案都无法同时实现 `add` 和 `delMin` 的高效操作。

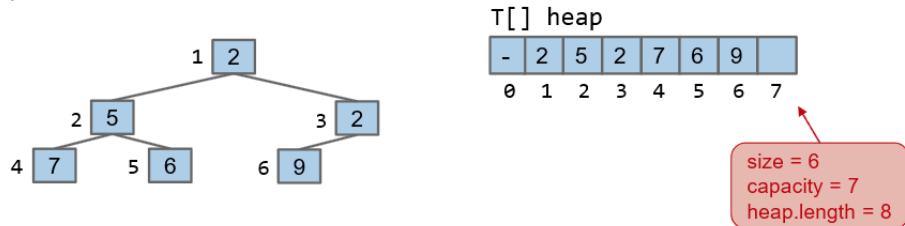
A binary heap is a binary tree that is complete and satisfies the heap property :

- Complete: balanced binary tree, could be left-aligned at the bottom level 结构平衡，除了最后一层外，所有层的节点都是满的；最后一层的节点从左到右连续排列，不出现空缺。
- Heap property: parent's item/priority is smaller or equal to the children's 任意父节点的值（或优先级）小于或等于其子节点的值。因此，整个堆的最小值始终位于根节点。

二叉堆的数组表示

Binary Heap Representation

- If we represent the binary heap by an array or an ArrayList, set the indices to start at 1, and order the nodes containing the item and the priority by the heap-property :
 - there is no explicit links needed between parent and the children
 - we can use the indices to access an item's parent or children
 - parent of an item at index k is $k/2$
 - children of an item at index k is $2*k$ and $2*k + 1$
- For example:



数组表示规则

- 数组下标从 **1** 开始（下标 0 可以留空或作为哨兵）。
- 完全二叉树的节点按层级顺序存入数组，无需额外的指针来表示父子关系。
- 利用下标计算父子节点：
 - 下标为 k 的节点，父节点下标为 $k/2$ （整数除法）
 - 下标为 k 的节点，左子节点下标为 $2*k$ ，右子节点下标为 $2*k + 1$

示例说明

图中的堆结构：

- 树节点按层级顺序存入数组：`[-, 2, 5, 2, 7, 6, 9, _, _]`
- `size = 6` 表示堆中实际存储了 6 个元素
- `capacity = 7` 表示数组能容纳的元素数量
- `heap.length = 8` 表示数组的总长度（含下标 0 的空位）

优势

- 无需显式的指针或链接，节省空间
- 父子节点的访问仅需简单的算术运算，效率高
- 完全二叉树的结构保证了数组存储的连续性，无内存碎片

Comparable Items and Swap

- Since the items are comparable, we can create a helper method to decide whether an item at index i is *greater than* the item at index j

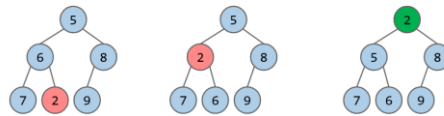
```
private boolean greater(int i, int j) {
    return ((Comparable<T>) heap[i]).compareTo(heap[j]) > 0;
}
```

- In the next slides, we will implement operations that swap (the reference to) the items

```
private void swap(int i, int j) {
    T temp = heap[i];
    heap[i] = heap[j];
    heap[j] = temp;
}
```

Restoring Heap Property : Swim

- What should we do if a child becomes smaller than its parent ?

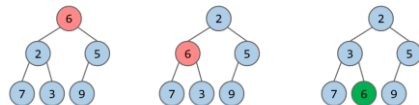


```
private void swim(int k) {
    while (k > 1 && greater(k/2, k)) {
        swap(k, k/2);
        k = k/2;
    }
}
```

swap child with the parent, until heap property is restored

Restoring Heap Property : Sink

- What should we do if a parent becomes larger than one child / both of its children ?



```
private void sink(int k) {
    while (2*k <= size) {
        int j = 2*k;
        if (j < size && greater(j, j+1)) j++;
        if (!greater(k, j)) break;
        swap(k, j);
        k = j;
    }
}
```

swap parent with the smaller child, until heap property is restored

Running Analysis of Binary Heap

How many comparisons?

- Item addition: $O(\text{height of a binary tree}) = O(\log N)$
- Minimum item removal: $O(2 \times \text{height of a binary tree}) = O(\log N)$

Representation	add	delMin
array	$O(1)$	$O(N)$
ordered array	$O(N)$	$O(1)$
binary heap	$O(\log N)$	$O(\log N)$

L9

Tree

- a set of nodes and a set of edges connecting nodes

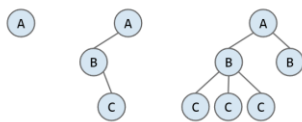
- there is exactly one path between any two nodes

节点集 + 边集，任意两点间唯一路径，无环连通图。

有根树(Rooted Tree)：指定根，除根外每个节点唯一父节点，无子节点为叶子。

Rooted Tree

- Rooted tree: a tree where we've chosen one node as the **root**
 - every node N except the root has **exactly one parent**, defined as the first node on the path from N to the root
 - a node with no **child** is called a **leaf**



- A: ROOT parent of B
- B: the parent of C
- C: the child of B

Rooted Tree

- A Java implementation: (BST: Binary Search Tree, more on it later in the course!)

```
public class BST<Key extends Comparable<Key>, Value> {
    private Node root; // root of BST

    private class Node {
        private Key key; // sorted by key
        private Value val; // associated data
        private Node left, right; // left and right subtrees
        private int size; // number of nodes in subtree

        public Node(Key key, Value val, int size) {
            this.key = key;
            this.val = val;
            this.size = size;
        }
    }
}
```

树的遍历 (Tree Traversal)

层次遍历 (Level Order Traversal) ----BFS

从上到下、从左到右的规则，按节点所在的层级依次访问

深度优先 DFS（前序、中序、后序）

depth first traversal: preorder, inorder, postorder （根的遍历顺序）

basic idea: traverse the deep nodes before the shallow ones 优先访问深层节点，再访问浅层节点

Visit: 访问节点内容 traverse: 探索节点但不访问内容

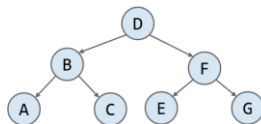
前序遍历（Preorder）规则：根节点 → 左子树 → 右子树

中序遍历（Inorder）规则：左子树 → 根节点 → 右子树

后序遍历（Postorder）规则：左子树 → 右子树 → 根节点

Tree Traversal and Level Order

- Sometimes we want to iterate over a tree, we call this **tree traversal**
 - there are many orders in which we might **visit** the nodes
- **Level Order**: visit top-to-bottom, left-to-right

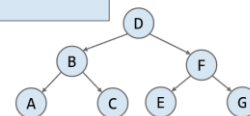


BFS 顺序: DBFACEG

Preorder Traversal

- **Preorder**: visit a node, then traverse left child and traverse right child
 - defined recursively

```
preorder(Node x) {  
    if (x == null) return;  
    print(x.key)  
    preorder(x.left)  
    preorder(x.right)  
}
```

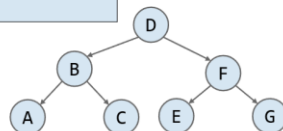


DFS 前序: DBACFEG

Inorder Traversal

- **Inorder**: traverse left child, visit a node, then traverse right child
 - defined recursively

```
inorder(Node x) {  
    if (x == null) return;  
    inorder(x.left)  
    print(x.key)  
    inorder(x.right)  
}
```

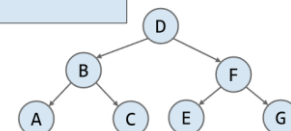


DFS 中序: ABCDEFG

Postorder Traversal

- **Postorder**: traverse left child, traverse right child, then visit a node
 - defined recursively

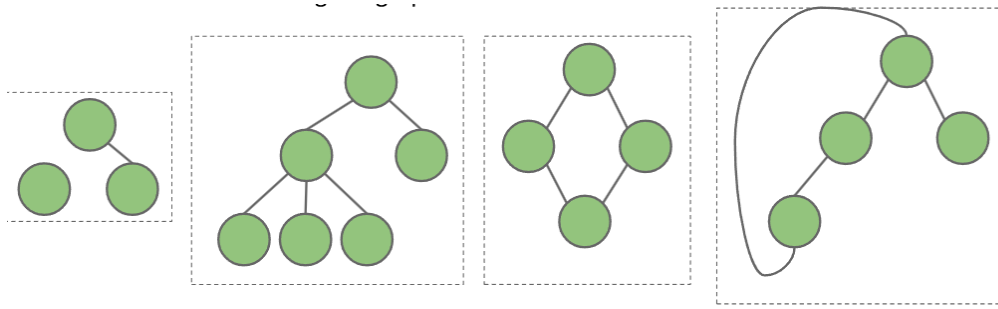
```
postorder(Node x) {  
    if (x == null) return;  
    postorder(x.left)  
    postorder(x.right)  
    print(x.key)  
}
```



DFS 后序: ACBEGFD

Graph

A graph consists of: a set of nodes , a set of zero or more edges, each of which connects two nodes 一组节点一组边，每条边可以连接两个节点，边的数量可以是 0 或者多个



四个都是图，注意和树定义的区别，树不可以有环

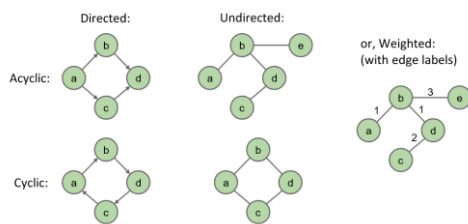
简单图 (Simple Graph)

无自环 (no loops): 不存在连接顶点到自身的边。

无平行边 (no parallel edges): 任意两个顶点之间，最多只有一条边。

Graph Types

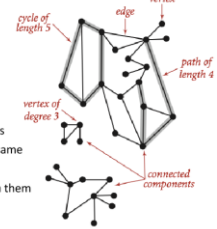
- Based on directionality, or whether there are cycles:



Acyclic 是无环

Graph Terminologies

- Graph:
 - set of **vertices (nodes)**
 - set of **edges** (pairs of vertices)
 - vertices with an edge between are **adjacent**
 - optional: vertices or edges may have **labels** (or **weights**)
- A **path** is a sequence of vertices connected by edges
 - a **simple path** is a path without repeated vertices
- A **cycle** is a path whose first and last vertices are the same
 - a graph with a cycle is **cyclic**
- Two vertices are **connected** if there is a path between them
 - if all vertices are connected, we say the graph is **connected**



Graph (图) 基础概念

由两部分组成:

- 顶点集 (vertices/nodes): 图中的点
- 边集 (edges): 顶点之间的连接 (每一条边是一对顶点)
- 若两个顶点之间有边，则称它们 **adjacent (相邻)**
- 可选: 顶点或边可以带有标签 / 权重 (labels/weights)

Path (路径)

- 定义: 由边连接起来的顶点序列
- simple path (简单路径)**: 顶点不重复出现的路径
- 图中标注的 **path of length 4**: 路径包含 4 条边 (对应 5 个顶点)

Cycle (环)

- 定义：起点和终点是同一个顶点的路径
- 含有环的图称为 **cyclic**（有环图）
- 图中标注的 **cycle of length 5**：环包含 5 条边（对应 5 个顶点）

Connected（连通性）

- 两个顶点之间如果存在路径，则称这两个顶点 **connected**（连通）
- 如果图中所有顶点都两两连通，则称这个图是 **connected graph**（连通图）
- 图中标注的 **connected components**（连通分量）：图中最大的连通子图（图里包含 2 个连通分量）

Degree（度）

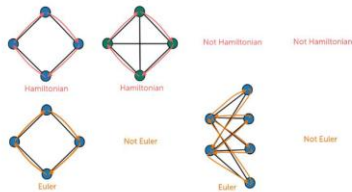
- 图中标注的 **vertex of degree 3**：这个顶点连接了 3 条边，它的度为 3

1	Graph Problems	图论问题
2	s-t Path. Is there a path between vertices s and t?	两点路径存在性问
3	Connectivity. Is the graph connected, i.e. is there a path between all vertices?	连通性问题：该图
4	Biconnectivity. Is there a vertex whose removal disconnects the graph?	双连通性问题：是
5	Shortest s-t Path. What is the shortest path between vertices s and t?	两点最短路径问题
6	Cycle Detection. Does the graph contain any cycles?	环检测问题：该图
7	Euler Tour. Is there a cycle that uses every edge exactly once?	欧拉回路问题：是
8	Hamilton Tour. Is there a cycle that uses every vertex exactly once?	哈密顿回路问题：
9	Planarity. Can you draw the graph on paper with no crossing edges?	平面性问题：能否
0	Isomorphism. Are two graphs isomorphic (the same graph in disguise)?	图同构问题：两个

点击图片可查看完整电子表格

Graph Problem Difficulty

- The problems may seem similar:
 - **Euler Tour.** Is there a cycle that uses every edge exactly once?
 - **Hamilton Tour.** Is there a cycle that uses every vertex exactly once?



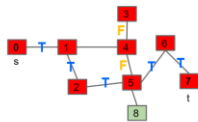
Graph Problem Difficulty

- The problems may seem similar:
 - **Euler Tour.** Is there a cycle that uses every edge exactly once?
 - **Hamilton Tour.** Is there a cycle that uses every vertex exactly once?
- Can we solve them efficiently?
 - an **efficient** Euler tour algorithm $O(\#edges)$ was found as early as 18'
 - despite decades of intense study, **no efficient algorithm** for a Hamilt exists, and best known algorithms are **exponential time**
 - in fact, if you could solve this (or show that there is no such efficient alg you win 1M USD for solving one of the **Millennium Prize Problems**

DFS

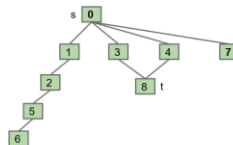
s-t Connectivity Algorithm

- connected(s, t).
 - mark s
 - does s == t? if so, return true
 - otherwise, if connected(v, t) for any unmarked neighbor v of s, return true
 - return false
- Assume using the order of values:



Depth First Traversal

- This idea of exploring a neighbor's entire subgraph before moving on to the next neighbor is known as **Depth First Traversal**
 - example: explore 1's subgraph completely before traversing the edge 0-3
 - called **depth first** because you go as deep as possible



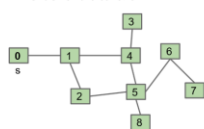
DepthFirstPaths Demo

Goal: Find a path from s to every other reachable vertex, visiting each vertex at most once. dfs(v) is as follows:

- Mark v
- For each unmarked adjacent vertex w:
 - set edgeTo[w] = v
 - dfs(w)

#	marked	edgeTo	
0	F	-	Start by calling dfs(0).
1	F	-	
2	F	-	
3	F	-	
4	F	-	
5	F	-	
6	F	-	
7	F	-	
8	F	-	

Order of dfs calls: 0



Order of dfs returns:

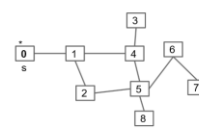
DepthFirstPaths Demo

Goal: Find a path from s to every other reachable vertex, visiting each vertex at most once. dfs(v) is as follows:

- Mark v
- For each unmarked adjacent vertex w:
 - set edgeTo[w] = v
 - dfs(w)

#	marked	edgeTo	dfs(0):
0	T	-	mark(0).
1	T	0	isMarked(1)? No.
2	T	1	• edgeTo[1] = 0. dfs(1).
3	T	4	No more children, so return.
4	T	5	
5	T	2	
6	T	5	
7	T	6	
8	T	5	

Order of dfs calls: 012543678



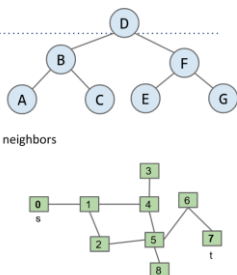
Order of dfs returns: 347685210

DFS 解决问题的 demo 在 ppt45 页左右自己看

Graph Traversals

- Recall the preorder tree traversal: DBACFEG

- What we just did in DepthFirstPaths is called **DFS Preorder**
 - visit (action, print) is **before** DFS calls to neighbors
 - the action was setting the array edgeTo
 - edgeTo[1] was set before DFS calls to neighbors 2 and 4
 - one valid DFS preorder for this graph: 012543678
 - equivalent to the order of dfs calls



Graph Traversals

- We could also do actions in **DFS Postorder**
 - visit (action, print) is **after** DFS calls to neighbors
 - dfs(s):
 - mark(s)
 - for each unmarked neighbor n of s, dfs(n)
 - print(s)
 - the result for dfs(0) would be: 347685210
 - equivalent to the order of dfs returns



Preorder

edgeTo[w] = v 这个动作，发生在对 w 调用 dfs(w) 之前

先访问再递归邻居

postorder

顺序和 DFS 函数的返回顺序完全一致（也就是递归栈的出栈顺序）

访问动作（比如打印、记录）发生在对邻居的所有 DFS 调用都返回之后

DFS 并不能保证找到的是最短路径（*对于无权图，“最短”指的是边数最少；对于带权图，指的是路径总权值最小。）

BFS

Breadth First Search

- Initialize a queue with a starting vertex s , mark that vertex, (and set dist to 0)
 - a **queue** is a list that has two operations: enqueue and dequeue
 - first in, first out (FIFO)
- Repeat until queue is empty:
 - remove vertex v from the front of the queue
 - for each unmarked neighbor n of v :
 - mark n
 - set $\text{edgeTo}[n] = v$ (and $\text{distTo}[n] = \text{distTo}[v] + 1$)
 - add n to end of queue

to track distance value

BFS (Breadth-First Search, 广度优先搜索) 的核心是按“距离”分层遍历, 所以必须用队列 (Queue) 来实现:

- 队列是一种“先进先出 (FIFO)”的数据结构, 正好能保证我们按距离从近到远处理节点。
- 它天然适合解决无权图的最短路径问题, 因为第一次访问到某个节点时, 走的就是边数最少的路径。

BreadthFirstPaths Demo

Goal: Find shortest path between s and every other vertex

- Initialize a queue with a starting vertex s , mark that vertex, set dist to 0
- Repeat until queue is empty:
 - remove vertex v from fringe
 - for each unmarked neighbor n of v : mark n , add n to queue, set $\text{edgeTo}[n] = v$, set $\text{distTo}[n] = \text{distTo}[v] + 1$

#	marked	edgeTo	distTo
0	F	-	-
1	F	-	-
2	F	-	-
3	F	-	-
4	F	-	-
5	F	-	-
6	F	-	-
7	F	-	-
8	F	-	-



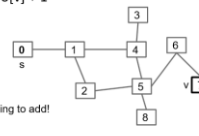
Queue: []

BreadthFirstPaths Demo

Goal: Find shortest path between s and every other vertex

- Initialize a queue with a starting vertex s , mark that vertex, set dist to 0
- Repeat until queue is empty:
 - remove vertex v from fringe
 - for each unmarked neighbor n of v : mark n , add n to queue, set $\text{edgeTo}[n] = v$, set $\text{distTo}[n] = \text{distTo}[v] + 1$

#	marked	edgeTo	distTo
0	T	-	0
1	T	0	1
2	T	1	2
3	T	4	3
4	T	1	2
5	T	2	3
6	T	5	4
7	T	6	5
8	T	5	4



Nothing to add!
Queue: []

BFS 例子 65 左右自己看谢谢

Erdős Number 厄多斯数 = 从厄多斯到该学者的最短合作路径的边数。BFS 解决 1200+

Graph Implementation

- How do we implement a graph?

```
public class Graph {
    public Graph(int V):           create empty graph with v vertices
    public void addEdge(int v, int w): add an edge v-w
    Iterable<Integer> adj(int v):  vertices adjacent to v
    int V():                       number of vertices
    int E():                       number of edges
    ...
}
```

- what is fixed in this implementation?
- what types of graphs are supported / not supported?

邻接表实现的无向无权图接口，顶点编号固定，不支持权重

Sample Method: Print Edges

- Using this definition, we can write:

```
public class Graph {
    public Graph(int V):           create empty graph with v vertices
    public void addEdge(int v, int w): add an edge v-w
    Iterable<Integer> adj(int v):  vertices adjacent to v
    int V():                       number of vertices
    int E():                       number of edges
    ...
}
```

- a method to print edges

```
1 2
2 1
2 3
3 2
4 1
```

```
public static void print(Graph G) {
    for (int v = 0; v < G.V(); v += 1) {
        for (int w : G.adj(v)) {
            System.out.println(v + " - " + w);
        }
    }
}
```



Graph Representation 1

$O(V^2)$ 邻接矩阵 $V \times V = V^2$

- Adjacency Matrix:** 邻接矩阵

2D boolean array

- Directed graph:

有向图是单向

s \ t	0	1	2
0	0	1	1
1	0	0	1
2	0	0	0



- Undirected graph:

Each edge is

represented twice

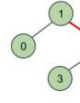
in the matrix

无向图 matrix[i][j]=1

matrix[j][i]=1 双向连通

邻接矩阵

v \ w	0	1	2	3
0	0	1	0	0
1	1	0	1	0
2	0	1	0	1
3	0	0	1	0



Graph Representation 2

$O(E)$ 只看 edge

- Edge Set** collection of all edges 边集

- example: `HashSet<Edge>`, where each Edge is a pair of ints

将边存为集合

$\{(0, 1), (0, 2), (1, 2)\}$



Graph Representation 3

$O(V+E)$

- Adjacency List:** an array of lists indexed by vertex number 邻接表

- most popular approach to represent graphs

一个数组，数组的每一个元素都是一个列表 index-顶点编号

列表-顶点所有的邻接点

0	[1, 2]
1	[2]
2	[0, 1]

bucket



超级无敌重要邻接表

Graph Representation Basic Operations

- Runtime of some basic operations for each representation:

Representation	添加边 	遍历 	打印 	查询 	空间 
Adjacency matrix	$O(1)$	$O(V)$	$O(V^2)$	$O(1)$	$O(V^2)$
Edge set	$O(1)$	$O(E)$	$O(E)$	$O(E)$	$O(E)$
Adjacency list	$O(1)$	$O(V)$	$O(V+E)$	$O(\text{degree}(v))$	$O(V+E)$

- In practice, adjacency lists are most common
 - many graph algorithms rely on $\text{adj}(v)$  频繁遍历
 - most graph algorithms are sparse (not too many edges in each bucket)  稀疏

Undirected Graph Implementation

```
public class Graph {
    private final int V;
    private List<Integer>[] adj;

    public Graph(int V) {
        this.V = V;
        adj = (List<Integer>[]) new ArrayList[V];
        for (int v = 0; v < V; v++) {
            adj[v] = new ArrayList<Integer>();
        }
    }

    public void addEdge(int v, int w) {
        adj[v].add(w); adj[w].add(v);
    }

    public Iterable<Integer> adj(int v) {
        return adj[v];
    }
}
```

array of list of integers

add to both buckets

Graph Algorithm Implementation

- A common design pattern in graph algorithms is that we decouple type from processing algorithm:
 - create a graph object
 - pass the graph to a graph-processing method (or constructor) in a client class
 - query the client class for information
- So in our DFS/BFS Paths case, we create a Paths class:

we don't add a new method to our Graph class

```
public class Paths {
    public Paths(Graph G, int s): Find all paths from G
    boolean hasPathTo(int v): is there a path from s to v?
    Iterable<Integer> pathTo(int v): path from s to v (if any)
}
```

解耦 (Decouple) 图的类型和处理算法

Graph 管存储, Paths 管算法, 代码清晰, 维护方便。

创建图对象: 使用通用的 Graph 类实例化一个图, 这个类只负责图的存储 (邻接表 / 矩阵), 不包含任何业务算法。

2. 传入图给算法类: 将 Graph 对象作为参数, 传入专门的算法类 (比如这里的 Paths 类) 的构造函数。

3. 从算法类查询结果: 通过算法类提供的方法 (如 hasPathTo、pathTo) 来获取路径信息。

好处 1: Graph 类非常稳定, 只负责存储, 不用为了每一个新算法 (DFS、BFS、拓扑排序、最短路径等) 都修改它。

好处 2: 算法被封装在独立的类中, 职责单一, 可复用性高。你可以为同一个 Graph 实现多种算法 (比如 DFSPaths 和 BFSPaths), 而互不干扰。

好处 3: 面向接口编程, 不同的算法实现都可以遵循 Paths 接口, 方便替换和扩展。

DepthFirstPaths Recursive Implementation (1)

```
public class DepthFirstPaths {
    private boolean[] marked;
    private int[] edgeTo;
    private int s;

    public DepthFirstPaths(Graph G, int s) {
        dfs(G, s);
    }

    private void dfs(Graph G, int v) {
        marked[v] = true;
        for (int w : G.adj(v)) {
            if (!marked[w]) {
                edgeTo[w] = v;
                dfs(G, w);
            }
        }
    }
}
```

marked[v] is true iff v connected to s
edgeTo[v] is previous vertex on path from s to v

data structure initialization
find vertices connected to s 初始PO

标记
遍历w
记录前驱 (递归回溯)

DepthFirstPaths Recursive Implementation (2)

```
...
public Iterable<Integer> pathTo(int v) {
    if (!hasPathTo(v)) return null;
    List<Integer> path = new ArrayList<>();
    for (int x = v; x != s; x = edgeTo[x]) {
        path.add(x);
    }
    path.add(s);
    Collections.reverse(path);
    return path;
}

public boolean hasPathTo(int v) {
    return marked[v];
}
```

获取完整路径
无路 return null
终点v到起点s 回溯
x = edgeTo[x]
x的前驱节点
是否连通

DepthFirstPaths Runtime Analysis

```
public class DepthFirstPaths {
    private boolean[] marked;
    private int[] edgeTo;
    private int s;

    public DepthFirstPaths(Graph G, int s) {
        dfs(G, s);
    }

    private void dfs(Graph G, int v) {
        marked[v] = true;
        for (int w : G.adj(v)) {
            if (!marked[w]) {
                edgeTo[w] = v;
                dfs(G, w);
            }
        }
    }
}
```

DFS 时间复杂度

Assume graph representation is adjacency list

vertex visits (no more than V calls) and initialisations
edge considerations (no more than 2E calls)
- 最近出现后V已邻接表

We sum:
• number of dfs calls: each vertex is visited at most once $O(V)$
• marked[w] checks: each edge is considered at most twice $O(E)$
so the runtime is $O(V + E)$

BreadthFirstPaths Implementation

```
public class BreadthFirstPaths {
    private boolean[] marked;
    private int[] edgeTo;

    public BreadthFirstPaths(Graph G, int s) {
        bfs(G, s);
    }

    private void bfs(Graph G, int s) {
        Queue<Integer> queue = new Queue<Integer>();
        queue.enqueue(s);
        marked[s] = true;
        while (!queue.isEmpty()) {
            int v = queue.dequeue();
            for (int w : G.adj(v)) {
                if (!marked[w]) {
                    queue.enqueue(w);
                    marked[w] = true;
                    edgeTo[w] = v;
                }
            }
        }
    }
}
```

marked[v] is true iff v connected to s
edgeTo[v] is previous vertex on path from s to v

顶点出入队 $O(V)$
set up starting vertex
遍历邻接点 $O(E)$
for freshly dequeued vertex v, for each neighbor that is unmarked:
- enqueue that neighbor to the queue
- mark it
- set its edgeTo to v

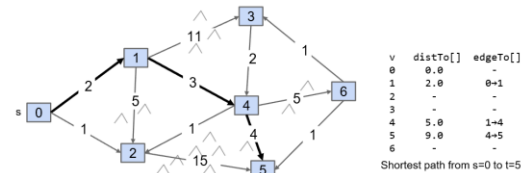
runtime is also $O(V + E)$ with similar arguments

L10

SPT Shortest Path Tree

Single Source Single Target Shortest Paths

Goal: Find the shortest paths from source vertex s to some target vertex t.

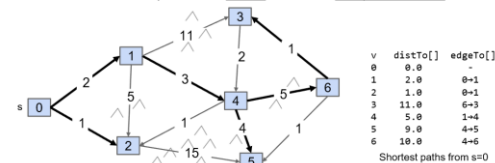


Observation: Solution will always be a **path with no cycles** (assuming non-negative weights)

单源单目标最短路径 点到点之间的最短距离 一条无环路径

Single Source Shortest Paths

Goal: Find the shortest paths from source vertex s to every other vertex



Observation: Solution will always be a **tree**
• Can think of as the union of the shortest paths to all vertices

单源最短路径 到其他所有点的最短路径 解一定是树

最短路径树 (Shortest Paths Tree, SPT) 是一棵以源点 s 为根的树, 它包含了从 s 到图中所有其他顶点的最短路径。SPT 中包含的边数为 $V-1$ 条。

Dijkstra 算法--SPT 单源最短路径 (非负权边)

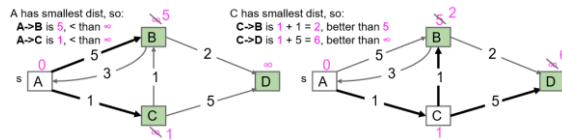
优先队列 (按距离源点由近到远) + relax

因为边权非负时，离源点更远的顶点，不可能通过后面处理的顶点，再得到比当前更短的路径。

Finding a Shortest Paths Tree 3

Dijkstra's Algorithm: Perform a **smallest distance first search**. When you visit v:

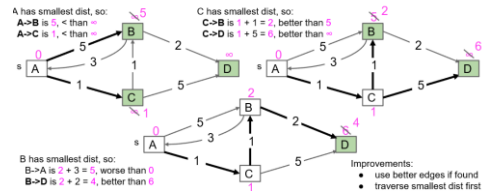
- For each from v to w, **relax that edge**



Finding a Shortest Paths Tree 3

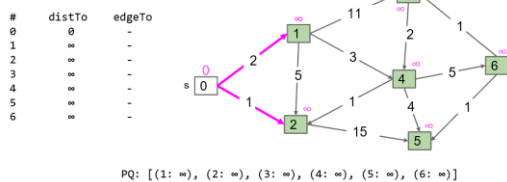
Dijkstra's Algorithm: Perform a **smallest distance first search**. When you visit v:

- For each from v to w, **relax that edge**



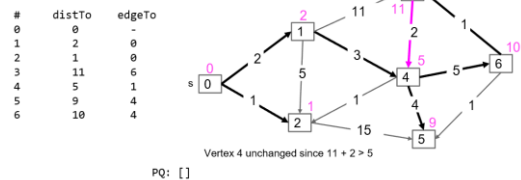
Dijkstra's Demo

Insert all vertices into PQ, storing vertices in order of distance from source
 Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v



Dijkstra's Demo

Insert all vertices into PQ, storing vertices in order of distance from source
 Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v



算法的伪代码

Dijkstra's Algorithm Pseudocode

Dijkstra's:

- PQ.add(source, 0)
- For other vertices v, PQ.add(v, infinity)
- While PQ is not empty:
 - p = PQ.delMin()
 - Relax all edges from p

Key invariants:

- edgeTo[v] is the best known predecessor of v
- distTo[v] is the best known total distance from source to v
- PQ contains all unvisited vertices in order of distTo

Relaxing an edge $p \rightarrow q$ with weight w:

- If $\text{distTo}[p] + w < \text{distTo}[q]$:
 - $\text{distTo}[q] = \text{distTo}[p] + w$
 - edgeTo[q] = p
 - PQ.decreaseKey(q, distTo[q])

Important properties:

- Always visits vertices in order of total distance from source
- Relaxation always fails on edges to visited (white) vertices

Guaranteed Optimality 最优性保证--证明思路

Guaranteed Optimality

Dijkstra's is guaranteed to be optimal so long as there are **no negative edges**

- Proof relies on the property that relaxation always fails on edges to visited (white) vertices

Proof sketch: Assume all edges have non-negative weights

- At start, $\text{distTo}[\text{source}] = 0$, which is optimal
- After relaxing all edges from source, let vertex v_1 be the vertex with minimum weight, i.e. that is closest to the source
Claim: $\text{distTo}[v_1]$ is optimal, and thus future relaxations will fail. Why?
 - $\text{distTo}[p] \geq \text{distTo}[v_1]$ for all p , therefore
 - $\text{distTo}[p] + w \geq \text{distTo}[v_1]$
- Can use induction to prove that this holds for all vertices after dequeuing

初始条件 • 一开始，源点的距离 $\text{distTo}[\text{source}] = 0$ ，这显然是最优的，因为到自己的最短路径就是 0。 2. 第一步处理 1. 松弛源点的所有出边，得到邻居的初始距离。 2. 取其中距离源点最近的顶点，记为 v_1 。

Claim (核心断言)：此时 $\text{distTo}[v_1]$ 已经是最优的，后续所有松弛操作都无法再缩短它的距离。

负权边不行：

Relaxation of already visited vertices can succeed 松弛操作可以对已经访问过的顶点生效。

Dijkstra's Algorithm Runtime

Priority Queue operation count, assuming binary heap MinPQ (lecture 8):

- add: V , each costing $O(\log V)$ time
- delMin: V , each costing $O(\log V)$ time
- decreaseKey: E , each costing $O(\log V)$ time

Overall runtime: $O(V \log V) + V \log V + E \log V$

- Assuming $E > V$, this is just $O(E \log V)$ for a connected graph

Operations	# Operations	Cost per operation	Total cost
PQ add	V	$O(\log V)$	$O(V \log V)$
PQ delMin	V	$O(\log V)$	$O(V \log V)$
PQ decreaseKey	E	$O(\log V)$	$O(E \log V)$

用 binary heap

Binary Heap with Decrease Key Implementation

- To maintain the logarithmic running time operations for our binary heap MinPQ implementation (lecture 8), we can create an **indexed priority queue** implementation
 - Start with same code as MinPQ
 - Maintain parallel arrays $\text{keys}[]$, $\text{pq}[]$, and $\text{qp}[]$ so that:
 - $\text{keys}[i]$ is the priority of i
 - $\text{pq}[i]$ is the index of the key in heap position i
 - $\text{qp}[i]$ is the heap position of the key with index i
 - Use $\text{swim}(\text{qp}[i])$ to implement **decreaseKey(i , key)**

索引优先队列

	数组名	作用
1		
2	$\text{keys}[i]$	存储顶点 i 的当前优先级 (也就是 Dijkstra 里的 dist)
3	$\text{pq}[i]$	存储 “堆位置 i 对应的顶点索引”
4	$\text{qp}[i]$	存储 “顶点 i 对应的堆位置”

点击图片可查看完整电子表格

MST Minimum Spanning Trees

Spanning Tree 生成树的定义

无向图 connected(联通) acyclic (无环) 包含所有顶点

A minimum spanning tree is a spanning tree of minimum total weight

最小生成树的应用

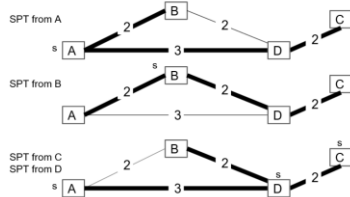
	English	中文翻译
1		
2	Old school handwriting recognition	老式手写识别
3	Medical imaging (e.g. arrangement of nuclei in cancer cells)	医学影像 (例如癌细胞中细胞核)
4	Max bottleneck paths	最大瓶颈路径问题
5	Real-time face verification	实时人脸验证
5	LDPC codes for error correction	用于纠错的 LDPC 码 (低密度)
7	Image registration with Renyi entropy	基于 Renyi 熵的图像配准
8	Find road networks in satellite and aerial imagery	从卫星和航拍图像中提取道路网络
9	Reducing data storage in sequencing amino acids in a protein	蛋白质氨基酸测序中的数据存储优化
10	Model locality of particle interactions in turbulent fluid flows	湍流流体中粒子相互作用的局域性模型

点击图片可查看完整电子表格

MST vs. SPT

Is the MST for this graph also a shortest paths tree? If so, using which node as the starting node for this SPT?

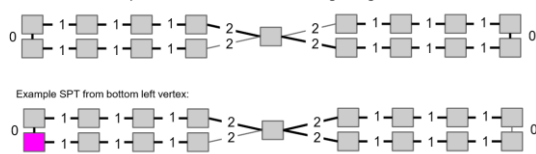
- A. A
- B. B
- C. C
- D. D
- E. No SPT is an MST



MST vs. SPT 2

Give a valid MST for the graph below.

- Is there a node whose SPT is also the MST?
- No!** Minimum spanning tree must include only 2 of the 2 weight edges, but the SPT always includes at least 3 of the 2 weight edges.



关于 MST 和 SPT

SPT 是单源路径最小，就是从指定的顶点出发遍历其他点得到的权重和最小，而 MST 则是全局成本最优

Cut Property 割性质

A cut is an assignment of a graph's nodes to two non-empty sets 就是把图里的所有顶点，分成两个非空的集合

A crossing edge is an edge which connects a node from one set to a node from the

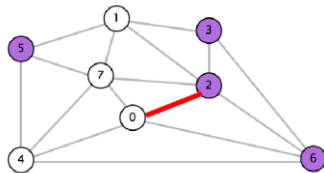
other set 连接两个不同集合顶点的边，就是跨边

对于任意一个割，权值最小的那条跨边，一定属于图的最小生成树（MST）。

Cut Property Example

Which edge is the minimum weight edge crossing the cut {2, 3, 5, 6}?

- 0-2. Must be part of the MST!

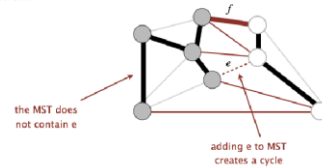


0-7	0.16
2-3	0.17
1-7	0.19
0-2	0.26
5-7	0.28
1-3	0.29
1-5	0.32
2-7	0.34
4-5	0.35
1-2	0.36
4-7	0.37
0-4	0.38
6-2	0.40
3-6	0.52
6-0	0.58
6-4	0.93

Cut Property Proof

Suppose that the minimum crossing edge e were not in the MST

- Adding e to the MST creates a cycle
- Some other edge f must also be a crossing edge
- Removing f and adding e is a lower weight spanning tree
- Contradiction!



证明：

假设 e 不在 MST 中我们设 MST 为 T ，它不包含最小跨边 e 。

加入 e 会形成环因为 T 是连通所有顶点的树，所以 e 的两个端点一定都在 T 里。把 e 加进去，就会形成一个环（Cycle）。

环中必然存在另一条跨边 f 这个环必须同时穿过割的两个集合，所以环里一定有另一条跨边 f ，连接两个集合。并且根据 e 是最小跨边的定义，我们有： $w(e) < w(f)$ 。

替换边得到更优的生成树我们做一个“换边操作”：

- 把 T 中的 f 删掉
- 把 e 加进去得到的新图依然是一棵生成树（连通、无环、包含所有顶点），但它的总权值是：总权值= $w(T) - w(f) + w(e)$ 因为 $w(e) < w(f)$ ，所以新的总权值比原来的 MST 更小。

矛盾！我们假设 T 是 MST（总权值最小），但现在找到了一棵权值更小的生成树，这就矛盾了。因此，我们最初的假设不成立，即： e 必须在 MST 中。

Generic MST Finding Algorithm 通用 MST 算法 一个算法框架

Start with no edges in the MST

初始化：一开始 MST 里一条边都没有。

Find a cut that has no crossing edges in the MST

找割：找到一个“割”（把顶点分成两个非空集合），并且 MST 里还没有任何跨边连接这两个集合。

Add smallest crossing edge to the MST

加边：把这个割中权值最小的跨边，加入 MST。

Repeat until V-1 edges

循环：重复步骤 2-3，直到 MST 里有 V-1 条边（V 是顶点数）。

理论可行，找 cut---Prim & Kruskal

Prim's Algorithm

选起点：从图中任意一个节点开始（Start from some arbitrary start node）

重复选边：不断选择一条端点一个在 MST 内、一个在 MST 外的权值最小的边，把它加入 MST（也就是图里说的 add shortest edge that has one node inside the MST under construction）

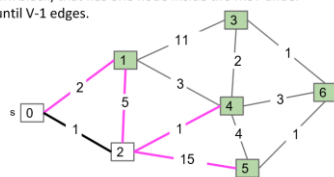
终止条件：重复步骤 2，直到 MST 中包含 V-1 条边（Repeat until V-1 edges）。

Prim's Demo

Start from some arbitrary start node.

- Add shortest edge (mark black) that has one node inside the MST under construction. Repeat until V-1 edges.

#	edgeTo
0	-
1	-
2	0
3	-
4	-
5	-
6	-

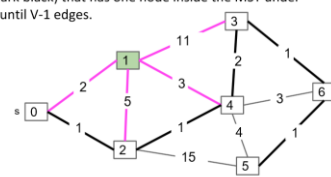


Prim's Demo

Start from some arbitrary start node.

- Add shortest edge (mark black) that has one node inside the MST under construction. Repeat until V-1 edges.

#	edgeTo
0	-
1	-
2	0
3	4
4	2
5	6
6	3



这边可以注意一下，是所有在 MST 里的点和所有不在里面的点的跨边，点的话还是挺多的，然后一直选权重最小的就行

最后肯定是选 1 啦哈哈哈哈哈哈然后 edgeTo[1]=0,还是根据的割性质，自动的把顶点分成了两个集合--在不在 MST 里

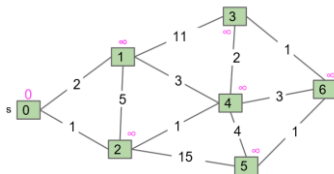
算法实现----优先队列 PQ 额可以具体看一下 ppt 的过程 p95 左右

Prim's Implementation Demo

Insert all vertices into PQ, storing vertices in order of distance from tree

Repeat: Remove closest vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0	0	-
1	∞	-
2	∞	-
3	∞	-
4	∞	-
5	∞	-
6	∞	-



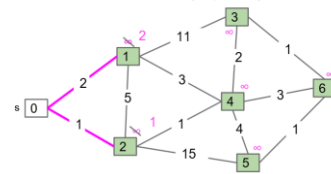
PQ: [(0: 0), (1: ∞), (2: ∞), (3: ∞), (4: ∞), (5: ∞), (6: ∞)]

Prim's Implementation Demo

Insert all vertices into PQ, storing vertices in order of distance from tree

Repeat: Remove closest vertex v from PQ, and relax all edges pointing from v

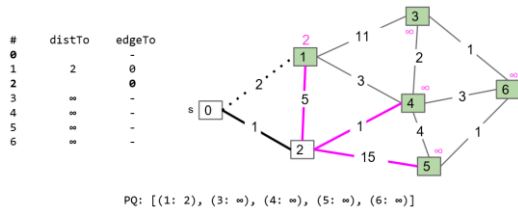
#	distTo	edgeTo
0	0	-
1	2	0
2	1	0
3	∞	-
4	∞	-
5	∞	-
6	∞	-



PQ: [(2: 1), (1: 2), (3: ∞), (4: ∞), (5: ∞), (6: ∞)]

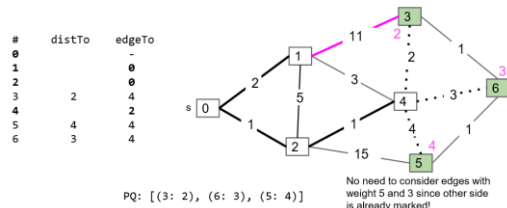
Prim's Implementation Demo

Insert all vertices into PQ, storing vertices in order of distance from tree
Repeat: Remove closest vertex v from PQ, and relax all edges pointing from v



Prim's Implementation Demo

Insert all vertices into PQ, storing vertices in order of distance from tree
Repeat: Remove closest vertex v from PQ, and relax all edges pointing from v



如果优化过的就出队列了 dist 也空着

松弛操作 (Relaxation) 这是两者最本质的区别: Dijkstra 的松弛: 更新的是「源点到该节点的最短路径长度」。条件是: $\text{distTo}[w] > \text{distTo}[v] + \text{weight}(v, w)$ 也就是, 走 源点 $\rightarrow v \rightarrow w$ 这条路, 比原来直接到 w 更短。

Prim 的松弛: 更新的是「该节点到 MST 的最小边权」。条件是: $\text{distTo}[w] > \text{weight}(v, w)$ 也就是, 直接连到 MST 里的 v , 比之前连到 MST 里的其他节点更便宜。

Prim's Implementation 1

```
public class PrimMST {
    public PrimMST(EdgeWeightedGraph G) {
        edgeTo = new Edge[G.V()];
        distTo = new double[G.V()];
        marked = new boolean[G.V()];
        pq = new MinPQ(G.V());

        distTo[s] = 0.0;
        setDistancesToInfinityExcept(s);
        insertAllVertices(pq);

        /* Get vertices in order of distance from tree */
        while (!pq.isEmpty()) {
            int v = pq.delMin();
            scan(G, v);
        }
        ...
    }
}
```

PQ is ordered by distTo tree, must be an indexed MinPQ with decreaseKey
get vertex closest to tree that is unvisited
scan means to consider all of a vertex's outgoing edges (next slide)

Prim's Implementation 2

```
while (!pq.isEmpty()) {
    int v = pq.delMin();
    scan(G, v);
}

private void scan(EdgeWeightedGraph G, int v) {
    marked[v] = true;
    for (Edge e : G.adj(v)) {
        int w = e.other(v);
        if (marked[w]) continue;
        if (e.weight() < distTo[w]) {
            distTo[w] = e.weight();
            edgeTo[w] = e;
            pq.decreaseKey(w, distTo[w]);
        }
    }
}
```

vertex v is closest, so add to MST
already in MST, so go to next edge
better path to a particular vertex found, so update current best known for that vertex

Prim's Runtime

```
while (!pq.isEmpty()) {
    int v = pq.delMin();
    scan(G, v);
}

private void scan(EdgeWeightedGraph G, int v) {
    marked[v] = true;
    for (Edge e : G.adj(v)) {
        int w = e.other(v);
        if (marked[w]) continue;
        if (e.weight() < distTo[w]) {
            distTo[w] = e.weight();
            edgeTo[w] = e;
            pq.decreaseKey(w, distTo[w]);
        }
    }
}
```

What is the runtime of Prim's algorithm?

- Assume all PQ operations take $O(\log V)$ time
- Give your answer in Big O notation

Prim's Algorithm Runtime

Priority Queue operation count, assuming binary heap based PQ:

- Addition: V , each costing $O(\log V)$ time
- Delete min: V , each costing $O(\log V)$ time
- Decrease key: $O(E)$, each costing $O(\log V)$ time

Overall runtime: $O(V \log V + V \log V + E \log V)$

- Assuming $E > V$, this is just $O(E \log V)$

Operation	# Operations	Cost per operation	Total cost
PQ add	V	$O(\log V)$	$O(V \log V)$
PQ delMin	V	$O(\log V)$	$O(V \log V)$
PQ decreaseKey	$O(E)$	$O(\log V)$	$O(E \log V)$

Kruskal's Algorithm

初始化: 把所有边标记为灰色 (未处理)。

按权值升序处理边: 从权值最小的边开始, 依次检查每一条边。

判断并加入 MST: 如果把这条边加入 MST 不会形成环, 就把它标记为黑色, 正式加

入 MST。如果会形成环，就直接跳过这条边。

终止条件：重复上述过程，直到 MST 中包含 $V-1$ 条边（ V 为顶点数），此时 MST 构建完成。

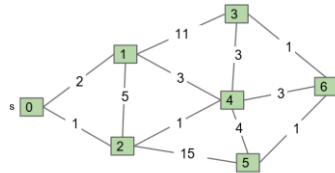
Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created.

- Repeat until $V-1$ edges

0-2 1
2-4 1
3-6 1
5-6 1
0-1 2
4-1 3
3-4 3
6-4 3
4-5 4
1-2 5
1-3 11
2-5 15

MST: []



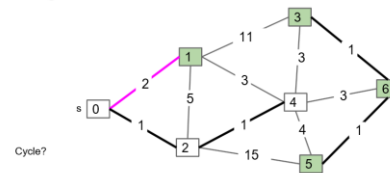
Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created.

- Repeat until $V-1$ edges

0-2 1
2-4 1
3-6 1
5-6 1
0-1 2
4-1 3
3-4 3
6-4 3
4-5 4
1-2 5
1-3 11
2-5 15

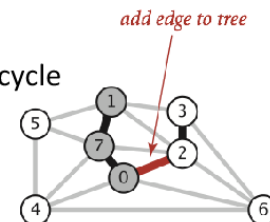
MST: [0-2, 2-4, 3-6, 5-6]



Kruskal's Algorithm Correctness

Initially mark all edges gray

- Consider edges in increasing order of weight
- Add edge to MST (mark black) unless doing so creates a cycle
- Repeat until $V-1$ edges



Why does Kruskal's work? Special case of generic MST algorithm

- Suppose we add edge $e = v \rightarrow w$
- Side 1 of cut is all vertices connected to v , side 2 is everything else
- No crossing edge is black (since we don't allow cycles)
- No crossing edge has lower weight (consider in increasing order)

how do we **implement** Kruskal's Algorithm?

正确性，依旧割性质

算法实现 141 左右自己看一下

• MinPQ using Binary Heap 二叉堆实现的最小优先队列 ---- 高效获取当前权值最小的边。

• Union Find using Weighted Quick Union with Path Compression data structures 带路径压缩的加权并查集 ---- 高效判断“加入边是否会形成环”。

Kruskal's Implementation Demo

Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

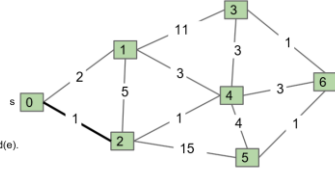
PQ: (2-4: 1),
(3-6: 1), (5-6: 1),
(0-1: 2), (4-1: 3),
(3-4: 3), (6-4: 3),
(4-5: 4), (1-2: 5),
(1-3: 11), (2-5: 15)

Removed edge: e=0-2

Cycle? No. union(0, 2). add(e).

UF: [0-2]

MST: [0-2]



Kruskal's Implementation Demo

Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

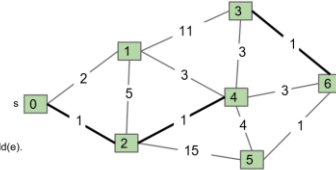
PQ: (5-6: 1),
(0-1: 2), (4-1: 3),
(3-4: 3), (6-4: 3),
(4-5: 4), (1-2: 5),
(1-3: 11), (2-5: 15)

Removed edge: e=3-6

Cycle? No. union(3, 6). add(e).

UF: [0-2-4, 3-6]

MST: [0-2, 2-4, 3-6]



Kruskal's Implementation

```
public class KruskalMST {
    private List<Edge> mst = new ArrayList<Edge>();

    public KruskalMST(EdgeWeightedGraph G) {
        MinPQ<Edge> pq = new MinPQ<Edge>();
        for (Edge e : G.edges()) {
            pq.add(e);
        }
        WeightedQuickUnionPC uf =
            new WeightedQuickUnionPC(G.V());
        while (!pq.isEmpty() && mst.size() < G.V()-1) {
            Edge e = pq.delMin();
            int v = e.from();
            int w = e.to();
            if (!uf.isSameGroup(v, w)) {
                uf.union(v, w);
                mst.add(e);
            }
        }
    }
}
```

Kruskal's Runtime

```
public class KruskalMST {
    private List<Edge> mst = new ArrayList<Edge>();

    public KruskalMST(EdgeWeightedGraph G) {
        MinPQ<Edge> pq = new MinPQ<Edge>();
        for (Edge e : G.edges()) {
            pq.add(e);
        }
        WeightedQuickUnionPC uf =
            new WeightedQuickUnionPC(G.V());
        while (!pq.isEmpty() && mst.size() < G.V()-1) {
            Edge e = pq.delMin();
            int v = e.from();
            int w = e.to();
            if (!uf.isSameGroup(v, w)) {
                uf.union(v, w);
                mst.add(e);
            }
        }
    }
}
```

What is the runtime of Kruskal's algorithm?

- Assume all PQ operations (Binary Heap MinPQ – Lecture 8) take $O(\log(V))$ time.
- Assume all UF-WQUPC (Union Find-Weighted Quick Union with Path Compression – Lecture 8 + Exercise 8) operations take $O(\log^* V)$ time
- Give your answer in Big O notation

Kruskal's Runtime

Kruskal's algorithm running time is $O(E \log E)$

Operation	Number of Times	Time per Operation	Total Time
add	E	$O(\log E)$	$O(E \log E)$
delMin	$O(E)$	$O(\log E)$	$O(E \log E)$
union	$O(V)$	$O(\log^* V)$	$O(V \log^* V)$
isSameGroup	$O(E)$	$O(\log^* V)$	$O(E \log^* V)$

If edges are already sorted, the running time is $O(E \log^* V)$

Question: Prim's runtime is $O(E \log V)$ and Kruskal's is $O(E \log E)$
Can we do better than them?

State-of-the-Art MST Algorithms

Year	Worst case	Discovered by
1975	$E \log \log V$	Yao
1984	$E \log^* V$	Fredman-Tarjan
1986	$E \log(\log^* V)$	Gabow-Gallil-Spencer-Tarjan
1997	$E \alpha(V) \log \alpha(V)$	Chazelle
2000	$E \alpha(V)$	Chazelle
2002	optimal	Pettie-Ramachandran
?	E	?

在边数远大于节点数的稠密图中，
Prim 通常更有优势。

Shortest Paths and MST Algorithms Summary

Problem	Algorithm	Runtime (if $E > V$)	Notes
Shortest Paths	Dijkstra's	$O(E \log V)$	Fails for negative weight edges
MST	Prim's	$O(E \log V)$	Analogous to Dijkstra's
MST	Kruskal's	$O(E \log E)$	Uses WQUPC

WQUPC =Weighted Quick Union with Path Compression 加权快速合并带路径压缩,
保证 find 和 union 的 O 几乎是常数

L11 图问题求解

无向图连通分量 (Connected Components)

Connectivity Queries

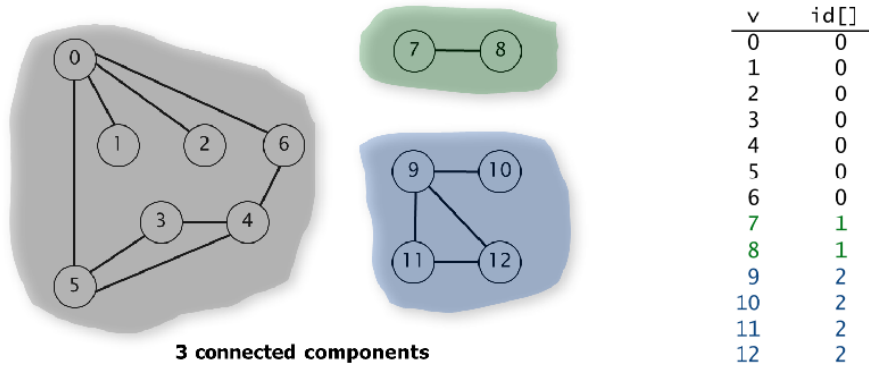
- Vertices v and w are **connected** if there is a path between them
- Goal: Preprocess graph to answer queries of the form **is v connected to w ?**
 - in **constant time**

<code>public class CC</code>	
<code>CC(Graph G)</code>	<i>find connected components in G</i>
<code>boolean connected(int v, int w)</code>	<i>are v and w connected?</i>
<code>int count()</code>	<i>number of connected components</i>
<code>int id(int v)</code>	<i>component identifier for v (between 0 and count() - 1)</i>

A connected component is a maximal set of connected vertices 一个连通分量，就是图中“互相连通的点的最大集合”。

Part 1: Connected Components

- A **connected component** is a maximal set of connected vertices
 - given connected components, can answer queries in **constant time**



判断 $id[v] == id[w]$; 一样则联通

连通性查询：我们通过一个预处理步骤，为每个顶点分配一个连通分量标识符 ($id[]$)
比较 $id[]$ 是否相等

查询是常数时间-- $id[]$ 是一个数组，数组访问操作是常数时间

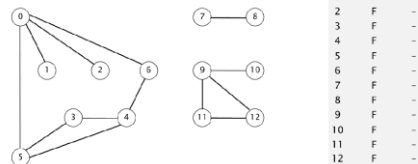
连通分量算法 ---DFS/BFS/Union-Find

Connected Components Algorithm

- Connected components algorithm:
 - initialize all vertices v as unmarked
 - for each **unmarked** vertex v , run **DFS** to identify all vertices discovered as part of the same component
 - To visit a vertex v :
 - mark vertex v as visited
 - recursively visit all unmarked vertices adjacent to v

Connected Components Demo

- To visit a vertex v :
 - Mark vertex v as visited
 - Recursively visit all unmarked vertices adjacent to v



Connected Components Implementation

```
public class CC {
    private boolean[] marked;
    private int[] id;
    private int count;

    public CC(Graph G) {
        marked = new boolean[G.V()];
        id = new int[G.V()];
        for (int v = 0; v < G.V(); v++) {
            if (!marked[v]) {
                dfs(G, v);
                count++;
            }
        }
    }

    private void dfs(Graph G, int v) {
        marked[v] = true;
        id[v] = count;
        for (int w : G.adj(v)) {
            if (!marked[w]) {
                dfs(G, w);
            }
        }
    }
}
```

run DFS from one vertex
in each component

all vertices discovered in
same call of DFS have same id

count 变量既是连通分量的编号，也是连通分量的计数器：它为每个分量分配唯一的标识，同时记录分量的总数，是实现常数时间连通性查询的关键。

优先级调度 (Precedence Scheduling)

优先级约束图，有向无环 DAG

拓扑排序 (Topological Sort) --DAG 排序算法

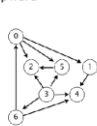
对一个有向无环图 (DAG)，将所有顶点排成一个序列，使得对于图中任意一条有向边 $u \rightarrow v$ ，顶点 u 一定出现在顶点 v 的前面。

Part 2: Topological Sort

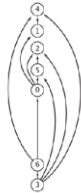
- Topological sort works on Directed Acyclic Graph (DAG)
 - redraw DAG so all edges point upward

0 → 5 0 → 2
0 → 1 3 → 6
3 → 5 3 → 4
5 → 2 6 → 4
6 → 0 3 → 2
1 → 4

directed edges



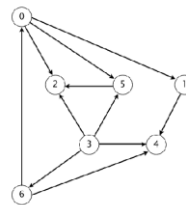
DAG



topological order

Topological Sort Demo

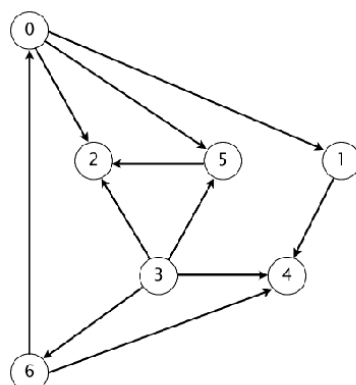
- Run DFS, return vertices in **reverse postorder**



- Visit 0, 1, 4
 - Done with 4, 1
 - Visit 2
 - Done with 2
 - Visit 5
 - Done with 5
 - Done with 0
 - Visit 3, 6
 - Done with 6, 3
- postorder

Topological Sort Demo

- Run DFS, return vertices in **reverse postorder**
 - when we are done with a vertex, put it in a stack, the pop-ed contents are in reverse postorder



postorder

4 1 2 5 0 6 3

topological order

3 6 0 5 2 1 4

Topological Sort Implementation

```
public class TopologicalSort {
    private boolean[] marked;
    private Stack<Integer> reversePostorder;

    public TopologicalSort(Graph G) {
        marked = new boolean[G.V()];
        reversePostorder = new Stack<>();
        for (int v = 0; v < G.V(); v++) {
            if (!marked[v])
                dfs(G, v);
        }
    }

    private void dfs(Graph G, int v) {
        marked[v] = true;
        for (int w : G.adj(v))
            if (!marked[w])
                dfs(G, w);
        reversePostorder.push(v);
    }
    ...
}
```

to store the topological sort,
create a method to return it

push the vertex v visited
by DFS into the stack

把 push(v) 放在 for 循环和 if 条件之外，能保证无论 v 有没有邻居、邻居是否被访问过，v 都会在其所有后继处理完毕后，被压入栈中。这完全符合拓扑排序算法对“后序遍历”的要求。

Strongly-connected Components ---有向图

强连通顶点 (Strongly Connected Vertices) 两个顶点 v 和 w 是强连通的，当且仅当：

- 存在一条从 v 到 w 的有向路径
- 同时也存在一条从 w 到 v 的有向路径简单说就是：两个点能互相到达。

强连通分量 (Strong Component) 一个强连通分量是图中最大的顶点子集，其中任

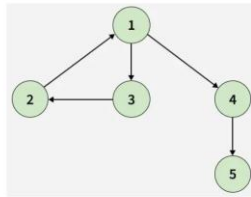
意两个顶点都强连通。

- “最大”的意思是：不能再往这个集合里加任何顶点了，否则就不满足“任意两点都能互相到达”的条件。

里面的节点只能和里面的节点互相到达，任何外面的节点，都不能和里面的节点形成双向路径。

In-Class Quiz 6

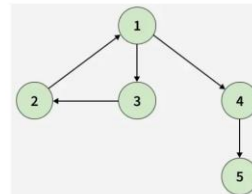
- How many **strongly-connected components** in the graph below? Please explain.



3 个

In-Class Quiz 7

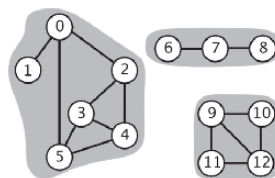
- If we reverse the direction of the edges in a graph, will we get the same **strongly-connected components**? Please explain.



原图中任意两个互相可达的点，在反转图中依然互相可达

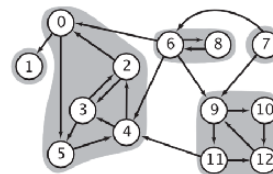
Connected components vs strongly-connected components

v and w are **connected** if there is a path between v and w



3 connected components

v and w are **strongly connected** if there is both a directed path from v to w and a directed path from w to v



5 strongly-connected components

connected component id (easy to compute with DFS)

	0	1	2	3	4	5	6	7	8	9	10	11	12
id[]	0	0	0	0	0	0	1	1	1	2	2	2	2

```
public boolean connected(int v, int w)
{ return id[v] == id[w]; }
```

constant-time client connectivity query

strongly-connected component id (how to compute?)

	0	1	2	3	4	5	6	7	8	9	10	11	12
id[]	1	0	1	1	1	1	3	4	3	2	2	2	2

```
public boolean stronglyConnected(int v, int w)
{ return id[v] == id[w]; }
```

constant-time client strong-connectivity query

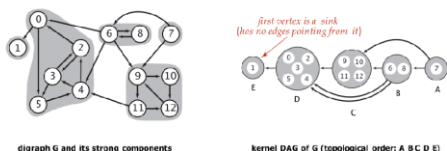
同一张图里，强连通分量的数量一定大于或等于连通分量的数量（要求更严格分类更多）

Kosaraju Algorithm's Intuition

有向图中用来求解强连通分量 (SCC) 的经典算法，依靠两次深度优先搜索 (DFS) 实现。

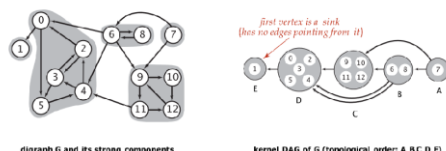
Kosaraju Algorithm's Intuition

- Notice that the strong components in G are same as in G^R (G 's reverse graph)
- Kernel DAG: contract each strong component into a single vertex



Kosaraju Algorithm's Intuition

- Notice that the strong components in G are same as in G^R (G 's reverse graph)
- Kernel DAG: contract each strong component into a single vertex
- Idea:
 - compute topological order (reverse postorder) in kernel DAG
 - run DFS, considering vertices in reverse topological order

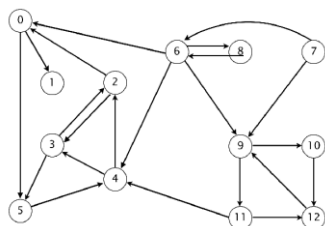


第一步：在反图 G^R 上做 DFS，得到逆后序。对 G^R 做 DFS，把节点按后序压入栈里。这个栈的顺序，就是缩点 DAG 的拓扑序的逆序。比如这张图里，栈弹出的顺序会是 E D C B A，也就是从汇点（sink）到源点（source）的顺序。

第二步：按栈的顺序，在原图 G 上做 DFS，每个树就是一个强连通分量。从栈顶（也就是拓扑序的起点）开始，在原图 G 上做 DFS。因为我们按“从汇点到源点”的顺序访问的，所以每次 DFS 能访问到的所有节点，都属于同一个强连通分量。比如从 E 开始 DFS，只能访问到它自己；再从 D 开始，能访问到整个 $\{0,2,3,4,5\}$ 分量，以此类推。

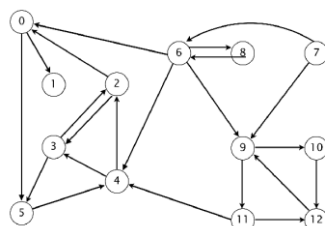
Kosaraju Algorithm Demo (1)

- Compute reverse postorder in G^R
 - 1 0 2 4 5 3 11 9 12 10 6 7 8



Kosaraju Algorithm Demo (2)

- Run DFS in G , visiting unmarked vertices in reverse postorder of G^R
 - 1 0 2 4 5 3 11 9 12 10 6 7 8



v	id[]
0	1
1	0
2	1
3	1
4	1
5	1
6	3
7	4
8	3
9	2
10	2
11	2
12	2

Kosaraju Algorithm Implementation

```
public class KosarajuSCC {
    private boolean[] marked;
    private int[] id;
    private int count;

    public KosarajuSCC(Graph G) {
        marked = new boolean[G.V()];
        id = new int[G.V()];
        TopologicalSort order = new TopologicalSort(G.reverse());
        for (int v : order.reversePostOrder()) {
            if (!marked[v]) {
                dfs(G, v);
                count++;
            }
        }
    }

    private void dfs(Graph G, int v) {
        marked[v] = true;
        id[v] = count;
        for (int w : G.adj(v)) {
            if (!marked[w]) {
                dfs(G, w);
            }
        }
    }
}
```

two lines difference compared to CC

Kosaraju 算法的时间复杂度是 $O(V + E)$ ，其中 V 是顶点数， E 是边数。

详细解释：

算法分为两个线性时间的阶段：

- 第一次 DFS（在反图 G^R 上）
 - 遍历反图 G^R 中所有顶点和边，生成逆后序排列。
 - 复杂度： $O(V + E)$ 。
- 第二次 DFS（在原图 G 上）
 - 按第一次得到的逆后序，在原图 G 上执行多次 DFS，每次找到一个强连通分量。
 - 这个过程同样遍历所有顶点和边一次。
 - 复杂度： $O(V + E)$ 。

结论

两次遍历加起来，总的时间复杂度为：

$$O(V + E) + O(V + E) = O(V + E)$$

L12 Binary Search Trees 二叉搜索树

Map

- Key-value pairs, also known as symbol tables, dictionary, associative arrays
 - **insert** a value with specified key
 - given a key, **search** for the corresponding value
 - for example: DNS search, given domain name, find corresponding IP address

domain name	IP address
www.princeton.edu	128.112.128.15
www.yale.edu	130.132.143.21
www.harvard.edu	128.103.060.55
www.simpsons.com	209.052.165.60

↑
key

↑
value

Map（符号表 / 字典）

定义：存储 key-value 键值对，支持快速按 key 查 value

典型操作：

- put (key, val)：插入 / 更新键值对
- get (key)：按键取值
- containsKey/containsValue/remove/keySet
- 应用：DNS 域名解析、词频统计等
- Java 实现：TreeMap（基于红黑树）、HashMap

	方法签名	作用说明	关键细节
1			
2	map.put(key, val)	添加 / 更新映射关系 key → val	若 key 已存在，会覆盖
3	map.get(key)	根据 key 获取对应的 value	若 key 不存在，返回 null
4	map.containsKey(key)	检查 Map 中是否存在指定 key	返回 boolean，时间复杂度为 O(log n) (TreeMap)
5	map.containsValue(value)	检查 Map 中是否存在指定 value	时间复杂度为 O(n) (HashMap)
6	map.remove(key)	删除 key 对应的键值对	返回被删除的 value (HashMap)
7	map.keySet()	返回 Map 中所有 key 组成的 Set	视图会随原 Map 同步

点击图片可查看完整电子表格

Map Example in Java (1)

- Suppose we have a list of strings of multiple duplicate words, such as ["a", "b", "a", "c", "b", "word", "a", "word"]
- We want to find out which words appear in the list and how many times each one appears
- Here is an algorithm NumWords using Map:
 - Create a Map<String, Integer>
 - Loop through all the strings in the list
 - Use each string as a key into the map
 - Use the Integer value to store the number of times that string has been seen
 - There are 2 cases:
 - the first time we see a word, it is not yet in the map, we add it
 - the later times we see the word, it is already in the map, we update it

Map Example in Java (2)

```
import java.util.HashMap;
import java.util.Map;

public class NumWords {
    public Map<String, Integer> wordCount(List<String> list) {
        Map<String, Integer> map = new HashMap<>();
        int num;
        for (String word : list) {
            if (!map.containsKey(word)) {
                map.put(word, 1);
            } else {
                num = map.get(word);
                map.put(word, num + 1);
            }
        }
        return map;
    }
}
```

key 为单词 value 为 num 有映射关系

the first time we've seen string word

we increment the number of word

In-Class Quiz 1

- Suppose we implement a map using an ordered array
 - two arrays, the elements in keys are sorted
 - what is the running time of insert? $O(n)$ 插入后需移动数组元素
 - what is the running time of search? $O(\log n)$ 二分查找

keys[]										vals[]									
0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9
A	C	E	H	L	M	P	R	S	X	8	4	12	5	11	9	10	3	0	7

Unordered Array

- To insert, need to shift all greater keys over (and the corresponding values)

key	value	keys[]										N	vals[]										
		0	1	2	3	4	5	6	7	8	9		0	1	2	3	4	5	6	7	8	9	
S	0	S											1	0									
E	1	E	S										2	1	0								
A	2	A	E	S									3	2	1	0							
R	3	A	E	R	S								4	2	1	3	0						
C	4	A	C	E	R	S							5	2	4	1	3	0					
H	5	A	C	E	H	R	S						6	2	4	1	5	3	0				
E	6	A	C	E	H	R	S						6	2	4	1	5	3	0				
X	7	A	C	E	H	R	S	X					7	2	4	1	5	3	0	7			
A	8	A	C	E	H	R	S	X					7	2	4	1	5	3	0	7			
M	9	A	C	E	H	M	R	S	X				8	2	4	1	5	9	3	0	7		
P	10	A	C	E	H	M	P	R	S	X			9	2	4	1	5	9	10	3	0	7	
L	11	A	C	E	H	L	M	P	R	S	X		10	2	4	1	5	11	9	10	3	0	7
E	12	A	C	E	H	L	M	P	R	S	X		10	2	4	1	5	11	9	10	3	0	7
		A	C	E	H	L	M	P	R	S	X		8	4	12	5	11	9	10	3	0	7	

entries in red were inserted

entries in black moved to the right

entries in grey did not move

circled entries are changed values

Rooted Binary Trees

先理解有根树 (Rooted Tree)

- 根 (Root)**: 有根树会指定一个节点作为起点, 叫做根节点。
- 父节点 (Parent)**: 除了根以外, 每个节点有且只有一个父节点, 它是“从当前节点往根走的路上遇到的第一个节点”。
- 叶子节点 (Leaf)**: 没有任何子节点的节点。

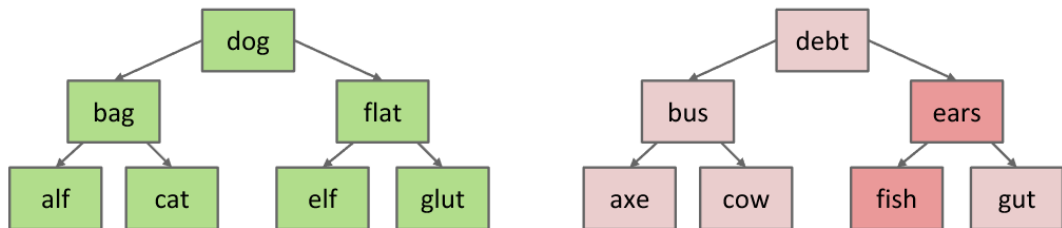
有根二叉树 (Rooted Binary Tree)

在有根树的基础上, 增加了一条关键限制:

每个节点最多只能有 **2** 个子节点 (可以是 0 个、1 个或 2 个)

Binary Search Trees

- A **binary search tree** is a rooted binary tree with the **BST property**, for every node X in the tree:
 - every key in the **left** subtree is **less** than X 's key
 - every key in the **right** subtree is **greater** than X 's key



对于树中的任意节点 X :

1. 左子树中所有节点的键值，都小于 X 的键值；
2. 右子树中所有节点的键值，都大于 X 的键值。

BST 键的排序规则（必须满足的三大性质）

为了实现“左小右大”的有序性，BST 的键必须支持一种合法的比较关系，满足以下三个条件：

性质	含义（对任意键 p 、 q 、 r ）
完全性 (Complete)	任意两个键都可以比较：要么 $p < q$ ，要么 $q < p$ ， $p = q$ 个成立。
传递性 (Transitive)	如果 $p < q$ 且 $q < r$ ，那么一定有 $p < r$ 。
反对称性 (Antisymmetric)	不存在 $p < q$ 和 $q < p$ 同时成立的情况。

点击图片可查看完整电子表格

在 Java 中，这意味着键必须实现 `Comparable` 接口（或使用 `Comparator`），这样才能调用 `compareTo()` 方法完成比较。

BST 不允许重复键

- 定义上，BST 的键是唯一的，不允许重复。
- 这是因为如果存在重复键，就无法满足上面的完全性（比如 $p == q$ 时， $p < q$ 和 $q < p$ 都不成立），也会破坏“左小右大”的结构规则。

插入重复键会发生什么？

当你插入一个已经存在的键，并附带一个新值时：

1. BST 会先找到这个键所在的节点；

2. 它不会新增一个节点，而是直接用新值覆盖掉原来的值。

- 这和 Map 的语义是一致的：键唯一，值可更新。

BST Node

- BST Node implementation:

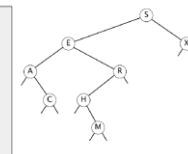
```
private class Node {
    private Key key;
    private Value val;
    private Node left, right;
    public Node(Key key, Value val) {
        this.key = key;
        this.val = val;
    }
}
```



Searching a Key in BST

- get: if key equals the key of node x: x.key, return the value
 - if key < x.key, search left subtree: x.left
 - if key > x.key, search right subtree: x.right

```
public Value get(Node x, Key key) {
    if (x == null)
        return null;
    if (key.equals(x.key))
        return x;
    else if (key < x.key)
        return get(x.left, key);
    else
        return get(x.right, key);
}
```



Insert Implementation

- put: associate a new value with key
- Search for key, then: if key in tree, update value; if key not in tree, add new node

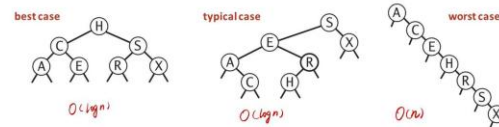
```
public void put(Key key, Value val) {
    root = put(root, key, val);
}

private Node put(Node x, Key key, Value val) {
    if (x == null) return new Node(key, val);
    int cmp = key.compareTo(x.key);
    if (cmp < 0)
        x.left = put(x.left, key, val);
    else if (cmp > 0)
        x.right = put(x.right, key, val);
    else if (cmp == 0)
        x.val = val;
    return x;
}
```

if the same key is found, update the value

Running Time of Search and Insert

- The number of compares for search/insert is equal to 1 + depth of node
 - depends on the tree shape
 - many BST corresponds to the same set of keys



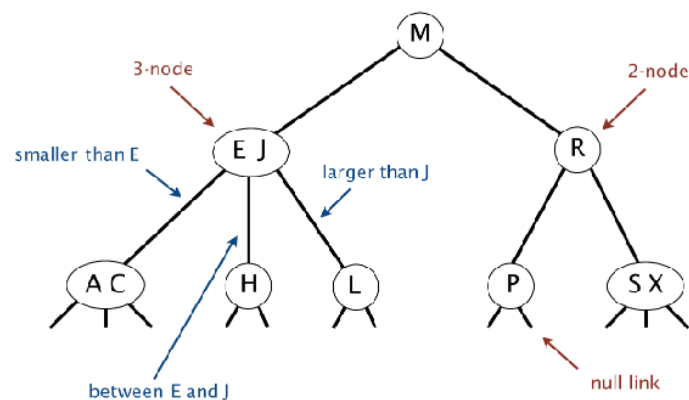
- tree shape depends on the order of insertion

$O(N)$ for search/insert in the worst case, can we do better?

2-3 Trees

2-3 Trees

- Allow 1 or 2 keys per node
 - 2-node**: one key, two children; and **3-node**: two keys, three children
- Symmetric order**: inorder traversal yields keys in ascending order
- Perfect balance**: every path from root to null link has same length



2-3 树的核心定义

1. 节点类型

- **2 - 节点**：包含 1 个键，拥有 2 个子节点（左、右），和普通二叉树节点类似。
- **3 - 节点**：包含 2 个键，拥有 3 个子节点，分别对应“小于第一个键”“介于两个键之间”“大于第二个键”的分支。图中 (E J) 就是典型的 3 - 节点，而 R 是 2 - 节点。

2. 核心性质

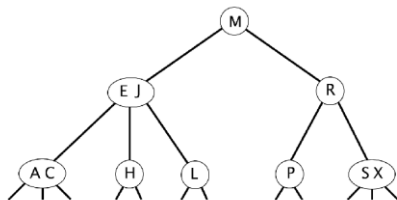
- **对称序（有序性）**：中序遍历 2-3 树，得到的键值序列是严格升序的，这和二叉搜索树的性质一致。
- **完美平衡**：从根节点到任意空链接（叶子节点下方的 null link）的路径长度都完全相同。这保证了树的高度始终为 $O(\log N)$ ，查找效率稳定。

图中结构解读

1. **根节点**：M（2 - 节点），将树分为左右两个子树。
2. **左子树根节点**：(E J)（3 - 节点）
 - 左孩子：(A C)，所有键都小于 E。
 - 中孩子：H，键值介于 E 和 J 之间。
 - 右孩子：L，键值大于 J。
3. **右子树根节点**：R（2 - 节点）
 - 左孩子：P，键值小于 R。
 - 右孩子：(S X)，所有键都大于 R。
4. **叶子节点**：如 (A C)、H、L、P、(S X)，它们的下方都连接着空链接，且从根到每个空链接的路径长度相同，体现了“完美平衡”。

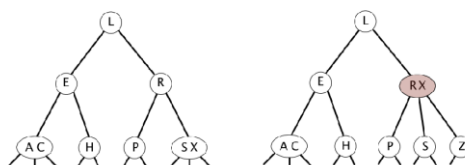
2-3 Tree Search

- Compare search key against keys in node
- Find interval containing search key
- Follow associated link (recursively)



2-3 Tree Insert (2)

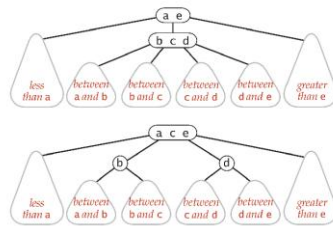
- Case 2: Insertion into a 3-node at bottom
 - Add new key to 3-node to create **temporary 4-node**
 - Move middle key in 4-node into parent; Repeat up the tree, as necessary
 - If you reach the **root** and it's a 4-node, **split** it into **three** 2-nodes



在 2-3 树中，只有当新键的插入导致根节点分裂为一个 4 - 节点时，树的高度才会增加 1。

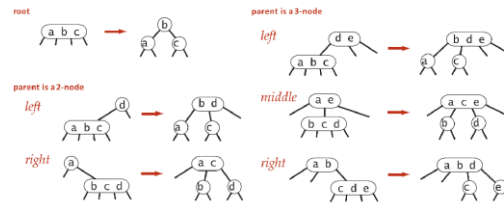
Local Transformations in 2-3 Tree

- Splitting a 4-node is a local transformation: constant number of operations



Global Properties in 2-3 Tree

- Invariants: maintains symmetric order and perfect balance
- Proof: each transformation maintains symmetric order and perfect balance



2-3 树的痛点

maintaining multiple node types is cumbersome 维护多种节点类型（2 - 节点、3 - 节点、临时 4 - 节点）十分繁琐

need multiple compares to move down tree 在树中向下查找时，需要进行多次键值比较

need to move back up the tree to split 4-nodes 分裂 4 - 节点时，需要回溯向上处理父节点

large number of cases for splitting 节点分裂的场景分支非常多

How to represent a 3-node? 如何表示 2-3 树中的 3 - 节点？

方案 1：普通二叉树（regular BST）

- no way to tell a 3-node from a 2-node 无法区分 3 - 节点和 2 - 节点
- cannot map from BST back to 2-3 tree 无法从二叉树还原出原来的 2-3 树结构

解读：普通二叉树只有父子关系，无法表达 2-3 树中“一个节点存两个键”的 3 - 节点，因此完全无法实现 2-3 树的逻辑。

方案 2：带“胶水节点”的普通二叉树（regular BST with "glue" nodes）

- wastes space, wasted link 浪费空间，浪费链接
- code probably messy 代码实现会很混乱

解读：额外增加一个“胶水节点”来连接两个键，模拟 3 - 节点。但这种方式会产生额外的节点和空链接，不仅占用更多内存，也会让后续的平衡、插入逻辑变得复杂，实际中几乎不会采用。

方案 3：带红色“胶水链接”的普通二叉树（regular BST with red

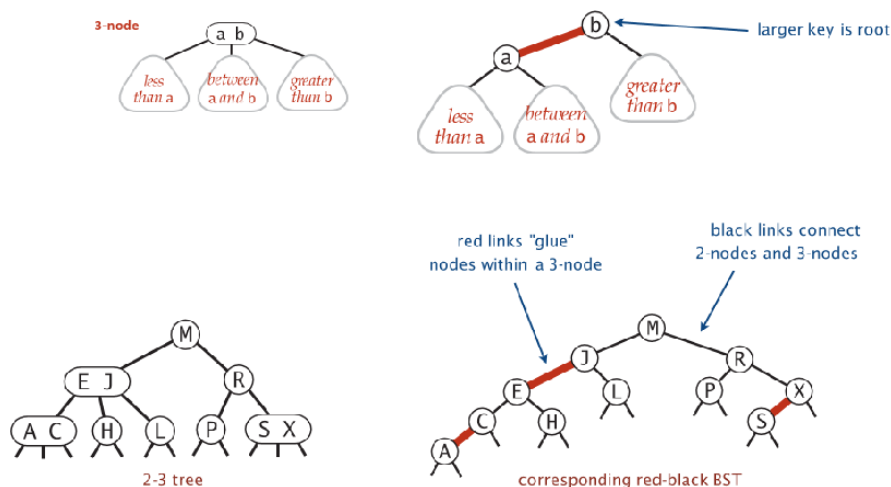
"glue" links)

- widely used in practice 实际中被广泛使用
- arbitrary restriction: red links lean left 人为限制: 红色链接必须向左倾斜 (左倾红黑树 LLRB 的核心规则)

左倾红黑树 (Left-Leaning Red-Black BSTs, LLRB)

Left-Leaning Red-Black BSTs

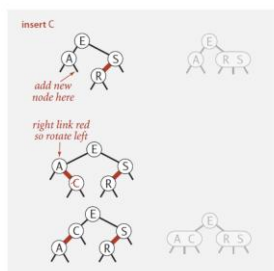
- Represent 2-3 tree as a BST, use *internal* left-leaning links as *glue* for 3-nodes



没有节点同时连接两条红色链接 从根节点到任意空链接的路径, 包含的黑色链接数量完全相同 红色链接必须左倾

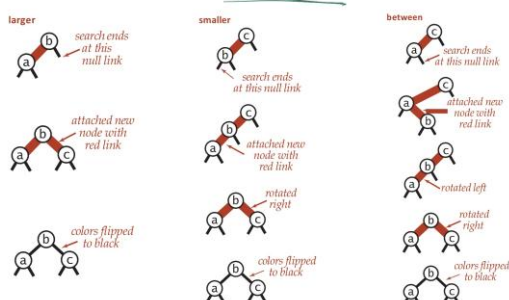
Insertion in a LLRB tree

- Case 1. Insert into a 2-node at the bottom. 在底部的2-节点插入新节点
- Do standard BST insert; color new link red. to maintain symmetric order and perfect black balance
 - If new red link is a right link, rotate left. to fix color invariants



Insertion in a LLRB tree

Warmup 2. Insert into a tree with exactly 2 nodes.

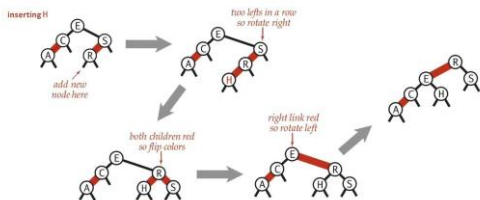


Insertion in a LLRB tree

在底部3-节点插入

Case 2. Insert into a 3-node at the bottom.

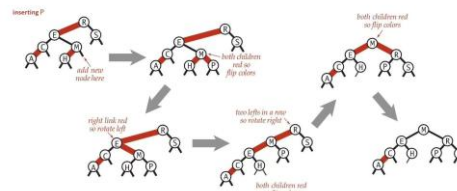
- Do standard BST insert; color new link red. to maintain symmetric order and perfect black balance
- Rotate to balance the 4-node (if needed).
- Flip colors to pass red link up one level. to fix color invariants
- Rotate to make lean left (if needed).



Insertion in a LLRB tree: passing red links up the tree

Case 2. Insert into a 3-node at the bottom.

- Do standard BST insert; color new link red. to maintain symmetric order and perfect black balance
- Rotate to balance the 4-node (if needed).
- Flip colors to pass red link up one level. to fix color invariants
- Rotate to make lean left (if needed).
- Repeat case 1 or case 2 up the tree (if needed).



L13

Last Lecture

- In **Lecture 12**, we have shown:



Data Structures	get(x)	put(x, val)	Notes
Ordered Arrays	$O(\log N)$	$O(N)$	Binary search, but need to shift
BSTs	$O(N)$	$O(N)$	Reduced to a chain in the worst case
2-3 Trees	$O(\log N)$	$O(\log N)$	Perfect balanced, but not a binary tree
Red-Black Trees	$O(\log N)$	$O(\log N)$	Perfect black balanced

In-Class Quiz 1

- Do you *agree* or *disagree* with this statement:

"Using BSTs or Red-Black Trees, we can store and search people's data with **their face images** as keys"

- **Explain** why you agree or disagree!

二叉搜索树 (BST) 和红黑树都要求键具有**全序关系** (即我们总能判断一个键大于、小于或等于另一个键)。人脸图像无法作为这些结构的有效键, 因为不存在一种有意义、可传递的方式来比较两张不同的人脸图像, 建立“大于”或“小于”的关系。没有定义好的比较规则, 就无法维持 BST 的有序性, 也无法执行插入和查找操作。而红黑树作为一种自平衡 BST, 同样受限于这一根本前提。

I disagree with this statement. The core issue is that both BSTs and Red-Black Trees require their keys to have a **total order** (i.e., we can always determine if one key is

greater than, less than, or equal to another key). Face images cannot serve as valid keys for these structures because there is no meaningful, transitive way to compare two different face images and establish a "greater than" or "less than" relationship. Without a defined comparison, the BST ordering property cannot be maintained, and insertions/searches cannot be performed. Red-Black Trees, being a type of self-balancing BST, face the same fundamental limitation.

Hashing

INTSet, 一种存储整数集合的朴素实现方式，核心思路是直接用数组下标代表数据本身。

INTSet

- Idea: Use an array of boolean that is indexed by the integer data
 - that is, the index is actually the data
 - initially all boolean values are false
 - when an item is added, set corresponding index to true

```
INTSet set;
set = new INTSet();
set.add(5);
set.add(2);
set.add(10);
```

contains(i) simply check whether value at index i is T or F

set contains 5, 2, 10

F	0
F	1
F	2
F	3
F	4
T	5
F	6
F	7
F	8
F	9
T	10
F	11
F	12
F	13
F	14
F	15

INTSet Implementation

- The implementation is very simple:

```
public class INTSet {
    private boolean[] items;

    public INTSet() {
        items = new boolean[2000000000];
    }

    public void add(int i) {
        items[i] = true;
    }

    public boolean contains(int i) {
        return items[i];
    }
}
```

roughly the number of integers

since accessing elements in an array is constant time, these operations = $O(1)$ time

INTSet Drawbacks

- There are some obvious problems with this implementation:

```
public class INTSet {
    private boolean[] items;

    public INTSet() {
        items = new boolean[2000000000];
    }

    public void add(int i) {
        items[i] = true;
    }

    public boolean contains(int i) {
        return items[i];
    }
}
```

1. extremely wasteful of memory! even if you only want to store a few integers you still need two billion booleans

2. you can only store integers need a way to generalize

essentially, when we can solve those two issues, we will get ourselves a hash table!

ASCII Characters

- In addition to lowercase letter, what if we want to store uppercase? numbers? special chars?
- We can use **ASCII encoding value** as the index
 - each char is assigned a value between 0 and 127: `char c = 'a'` is equiv with `char c = 97` in Java

characters 33-126 are printable

the others do other things, like the new line character

33 !	49 1	65 A	81 Q	97 a	113 o
34 "	50 2	66 B	82 R	98 b	114 p
35 #	51 3	67 C	83 S	99 c	115 q
36 \$	52 4	68 D	84 T	100 d	116 r
37 %	53 5	69 E	85 U	101 e	117 s
38 &	54 6	70 F	86 V	102 f	118 t
39 '	55 7	71 G	87 W	103 g	119 u
40 (	56 8	72 H	88 X	104 h	120 v
41)	57 9	73 I	89 Y	105 i	121 w
42 *	58 :	74 J	90 Z	106 j	122 x
43 +	59 ;	75 K	91 [	107 k	123 y
44 ,	60 <	76 L	92 \	108 l	124 z
45 -	61 =	77 M	93]	109 m	
46 .	62 >	78 N	94 ^	110 n	
47 /	63 ?	79 O	95 _	111 o	
48 0	64 @	80 P	96 `	112 p	

从朴素想法到哈希表：一步步演进

INTSet: 最朴素的整数集合（直接寻址）

- 思路：用布尔数组，数据本身当下标
- add(i): `items[i] = true`
- contains(i): 直接返回 `items[i]`
- 优点：真 $\cdot O(1)$
- 致命缺点：
 - 极度浪费内存（存几个数也要开 20 亿数组）
 - 只能存整数

ENGSet: 存英文单词

- 思路：把字符串转成整数（26 字母 → 27 进制）
- $\text{cat} = 3 \times 27^2 + 1 \times 27 + 20 = 2234$
- 问题：
 - 只用首字母 → **严重冲突**（cat / cow 都到 3）
 - 全字符计算 → **数字爆炸、Java int 溢出**
 - overflow 溢出 → 非法下标 + 冲突

通用哈希思想诞生

- 任意类型（字符串、对象、中文）→ 哈希函数 → **哈希码（Hash Code）** 哈希码的可能取值最多只有 42 亿多种。
- 哈希码 → 取模 → 合法数组下标
- 核心：把无限集合映射到有限整数范围

核心概念：哈希与冲突两大挑战

How do we resolve the hash code collisions? 我们如何解决哈希码冲突?

let us call this collision handling 我们称之为**冲突处理（collision handling）**

How do we compute a hash code for arbitrary objects? 我们如何为任意对象计算哈希码?

let us call this computing a hash code 我们称之为**计算哈希码（computing a hash code）**。

哈希（Hashing）

- Hash Function 哈希函数**：任意键 → 固定范围整数（哈希码）
- Hash Code 哈希码**：映射后的整数，用作数组下标

冲突（Collision）—— 不可避免！

- 定义：两个不同键得到**同一个哈希码 / 下标**
- 原因：**鸽巢原理（Pigeonhole Principle）**
 - 键无限多，哈希码只有 2^{32} 种
 - 必然有多个键共用一个位置

本课唯一冲突解法：**Separate Chaining 分离链地址法**

- 每个下标位置不存单个值，存一个**链表 / 桶（Bucket）**

****如何实现一个桶？ (How to implement a bucket?)**

use a linked list: 使用链表实现

could use an array list: 也可以用动态数组实现

could also use an array set: 还可以用集合实现

any data structure which you can search and iterate: 任何支持查找和遍历的数据结构都可以作为桶 ******

- 冲突元素都放进同一个桶里
- 查找：先定位桶，再在桶里遍历查找

哈希表完整工作流程（必背）

1. 用 `hashCode()` 算出键的哈希码
2. 用 `floorMod(哈希码, 桶数 M)` 得到合法下标（解决负哈希码问题）
3. 在下标对应的**链表**里添加 / 查找元素
4. 元素太多 → **动态扩容 (Resizing)** → 重算所有下标

Separate Chaining

- In separate chaining, when an item x gets added at index h :
 - If bucket h is empty, we create a new list containing x and store it at index h
 - If bucket h is already a list, we add x to this list *if it is not already present*

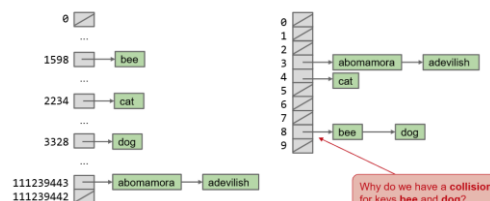
`add("a")`
`add("abomamora")`



using `englishToInt`,
`abomamora` hashes to
111239443, create new list

In-Class Quiz 4

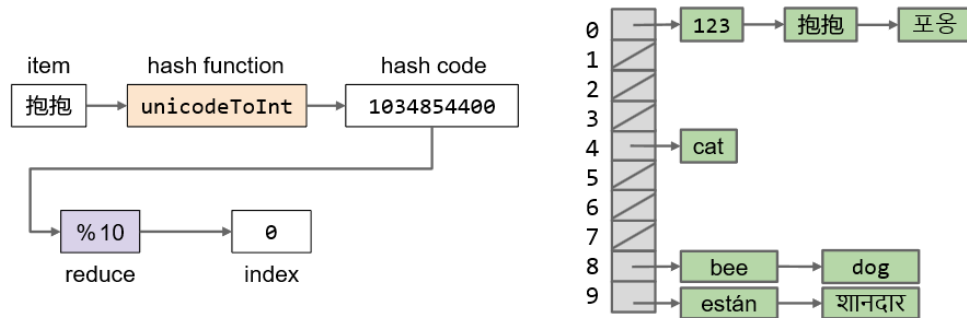
- We could reduce the number of buckets, and use modulus of hash code to compute index
 - for example, use 10 buckets, and compute bucket number = hash code % 10



取模后在一个 bucket 里面

Hash Table

- We have just created the so-called **hash table** !
 - The item is converted by a **hash function** into an integer representation called a **hash code**
 - The hash code is then reduced to a **valid index**, usually using the modulus operator



先用哈希函数把任意数据变成整数，再用取模运算把这个整数压缩成数组下标，然后用链地址法处理冲突。